Parallel Computer Architecture

Lecture 2:

- Flynn's Taxonomy
- Intuition for Parallelism

Armin Mehrabian

Spring 2025

Taxonomies and Definitions

Outline

- Definitions and Conceptual Classifications
  - Parallel Processing, MPP's, and Related Terms
  - Flynn's Classification of Computer Architectures
- Operational Models for Parallel Computers
- Interconnection Networks
- MPP's Performance

Definitions and Conceptual Classification

What is Parallel Processing?

- A data processing approach that emphasizes exploring and exploiting **inherent parallelism** in a problem.

Related Terms:

- **Massively Parallel Processors (MPPs):** Systems designed with many processors working concurrently.
- **Scalable Processors:** Architectures that maintain efficiency as more processors are added.
- **Heterogeneous Processing:**
  - 1990s: Workstations with processors from different vendors.
  - Now: Use of accelerators like **GPUs, FPGAs, TPUs**, and other specialized hardware (e.g., Intel Xeon Phi, until its discontinuation).
- **Grid Computing:** Distributed systems across multiple administrative domains for large-scale computing.
- **Cloud Computing:** Virtualized resources on demand over the internet.

Definitions and Conceptual Classification

Why Massively Parallel Processors (MPPs)?

- **Increased Processing Speed & Memory:** Enables the study of problems with **higher resolutions** or **larger datasets**.
- **Cost Efficiency:**
  - Offers a **low-cost alternative** to traditional vector machines.
  - Achieves high performance without relying on expensive processors or memory technologies.

Vector Machines

Vector Machines: specialized computing systems designed to handle **vector operations** efficiently. They were widely used in high-performance computing (HPC) during the 1970s and 1980s, especially for scientific and engineering applications. Here's a quick overview:

Key Characteristics of Vector Machines:

1. **Vector Processing:**
   - Instead of processing one data element at a time (like scalar processors), vector machines operate on entire vectors (arrays of data) simultaneously.
   - This is ideal for operations like matrix multiplications, simulations, and other computations that involve repetitive data patterns.
2. **Hardware Design:**
   - These machines use specialized vector registers and pipelines optimized for handling long sequences of data.
   - Examples include Cray-1 and the NEC SX series.
3. **High Performance for Specific Applications:**
   - Vector machines excel in tasks with predictable data patterns, like weather modeling, fluid dynamics, and structural analysis.
4. **Cost and Complexity:**
   - Vector machines were extremely expensive to build and maintain.
   - Their hardware was less adaptable to general-purpose computing

Vector vs. MPP

Architectures in Supercomputers:

1. **Vector Processor-Based Systems:**
   - Specialized for high-speed computation on vector data.
   - Examples: Cray X-MP, Y-MP, C90, T94.
   - **Key Features:** Few, highly optimized processors; limited scalability.

2. **Microprocessor-Based Systems (MPP):**
   - Built using many off-the-shelf microprocessors.
   - Examples: IBM Blue Gene, Thinking Machines CM-5.
   - **Key Features:** Highly scalable and cost-efficient.

Historical Transition:

- 1980s–90s: Shift from vector processors to microprocessor-based architectures due to scalability and cost benefits.

Von Neumann Architectures & Flynn's Taxonomy

Von Neumann Architecture

Key Concept:

- **Separation of Memory and Compute:**
  - Memory stores both instructions and data.
  - The processor fetches instructions and data from memory to execute tasks.

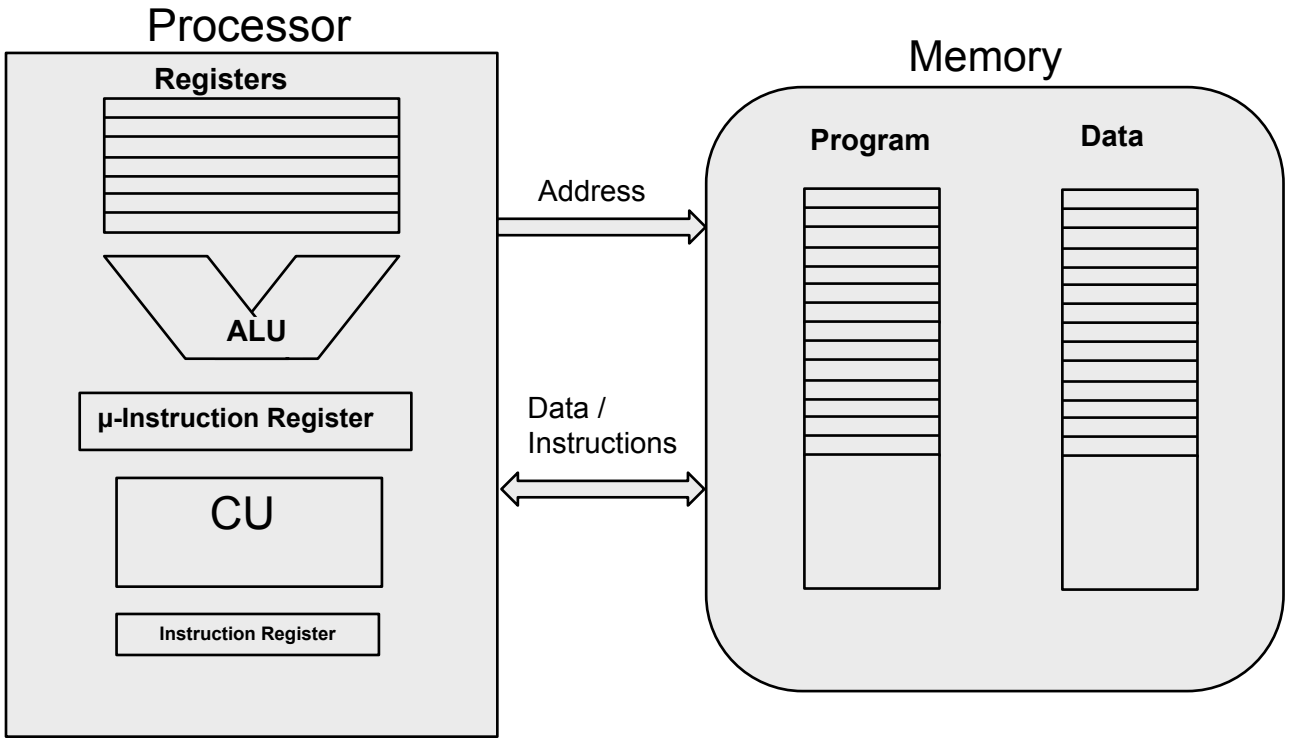
Core Features:

1. **Single Memory:** Shared storage for both program instructions and data.
2. **Sequential Execution:** Operations are performed step-by-step in a cycle:
   - Fetch → Decode → Execute.
3. **Control by a Central Processor:** Manages all computation and memory operations.

Why It Matters:

- Forms the foundation of most modern computers.
- Simple and flexible design, but limited by the **Von Neumann Bottleneck** (slow memory access compared to processor speed).

Von Neumann Architecture



μ-Instructions

Micro-instructions are low-level control instructions used in **microprogrammed control units** within a CPU. These instructions break down complex machine-level instructions into smaller steps that directly control the hardware.

Example of Micro-Instructions

Consider a simple example of executing an **ADD instruction** at the machine level (e.g., adding the contents of two registers). The micro-instructions to implement this could look something like this:

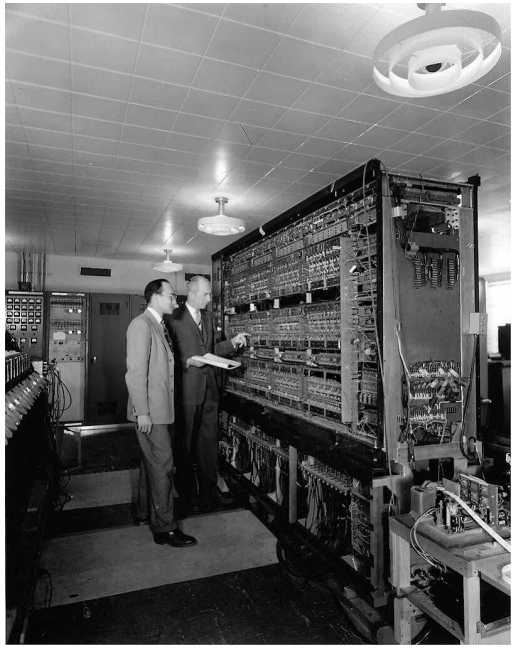
1. **Fetch Phase:**
   - `MAR ← PC` (Move the value of the Program Counter into the Memory Address Register to locate the next instruction).
   - `MDR ← MEM[MAR]` (Load the instruction from memory into the Memory Data Register).
   - `IR ← MDR` (Move the instruction into the Instruction Register).
   - `PC ← PC + 1` (Increment the Program Counter to the next instruction).
2. **Decode Phase:**
   - `Control Unit interprets the opcode in IR`.
   - Determine the operation (e.g., ADD) and operands (e.g., contents of registers R1 and R2).
3. **Execute Phase:**
   - `TEMP ← R1` (Move the contents of Register R1 into a temporary register).
   - `ALU_OUT ← TEMP + R2` (Perform the addition in the Arithmetic Logic Unit).
   - `R3 ← ALU_OUT` (Store the result into the destination register, R3).
4. **Write Back Phase (if needed):**
   - Update flags in the **Status Register** (e.g., carry, overflow, zero) based on the result.

The IAS Machine and Von Neumann Architecture

- **Developed by:**
  - John von Neumann at the Institute for Advanced Study (IAS), Princeton.
- **Historical Significance:**
  - First electronic computer to implement the **stored program concept.**

Key Features:

1. **Stored Program Concept:**
   - Programs and data are stored together in the same memory.
2. **Von Neumann Architecture:**
   - Common memory shared for both instructions and data.
3. **Instruction Cycle:**
   - **Fetch** → **Decode** → **Execute** → **Write Back** (basic operational steps of the CPU).



Flynn's Classification (1966)

- A simple and memorable way to classify computer architectures.
- Multiplicity of instruction streams **(SI/MI)** and data streams **(SD/MD).**

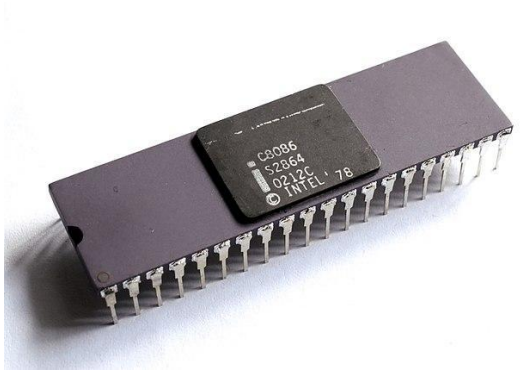
1. Single Instruction, Single Data (SISD)

Definition:

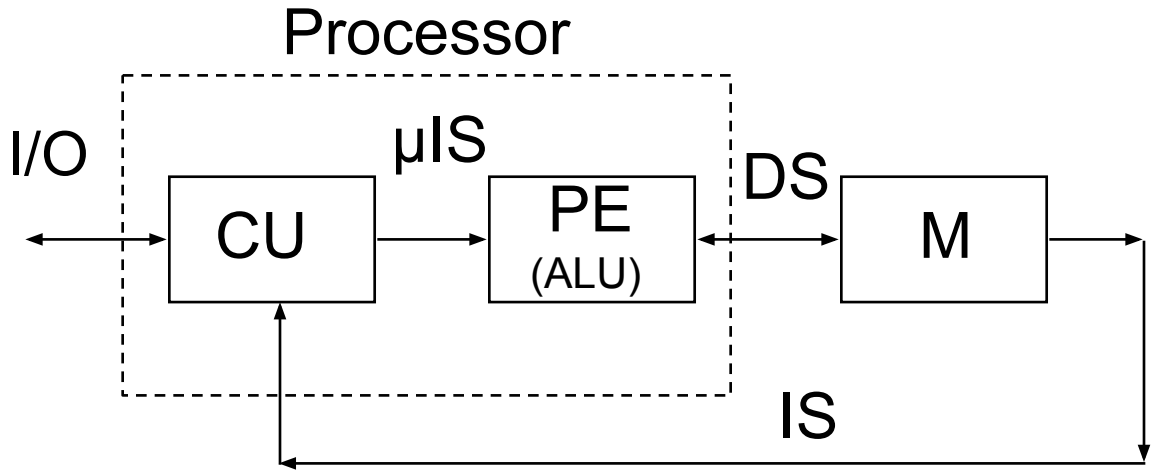
- A single control unit fetches and executes one instruction at a time, operating on a single data item.
- Traditional sequential computing.

Examples:

- **Classical Example:**
  - Early computers like the **Intel 8086** or **ENIAC**.
- **Modern Example:**
  - **Single-core CPUs** when running a single thread (e.g., a single-core processor executing simple tasks).
- **Use Case:**
  - Tasks with no inherent parallelism, such as simple control programs.



1. Single Instruction, Single Data (SISD)



CU= Control Unit
PE= Processing Element (same as ALU)
M= Memory
IS= Instruction Stream
DS= Data Stream,
μIS= MicroInstructions Stream

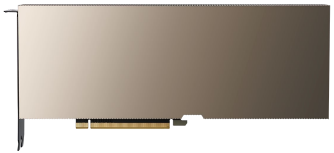
2. Single Instruction, Multiple Data (SIMD)

Definition:

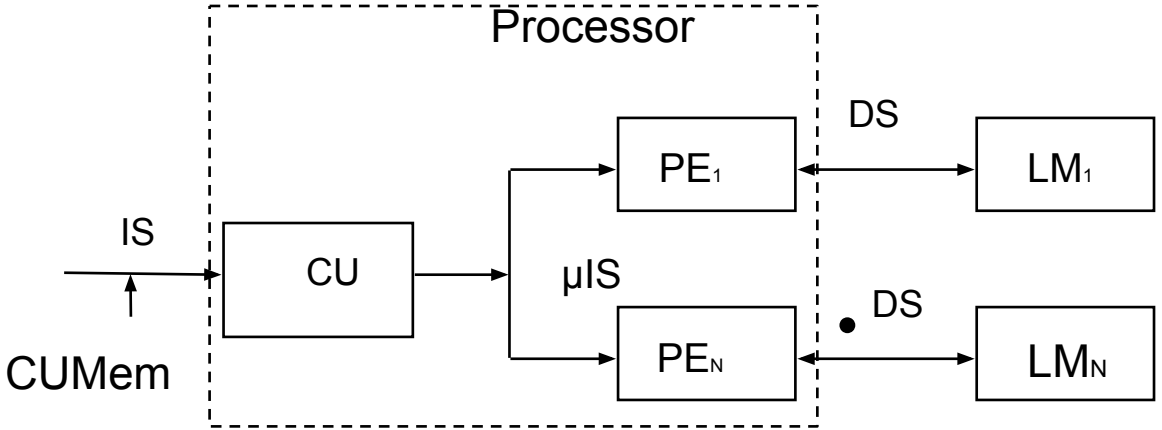
- A single instruction operates on multiple data elements simultaneously. Ideal for data-level parallelism.
- Used in vector processors and GPUs.

Examples:

- **Classical Example:**
  - **Cray-1** or early **vector processors**.
- **Modern Example:**
  - **GPUs:**
    - NVIDIA GPUs (e.g., CUDA cores processing image pixels simultaneously).
    - AMD Radeon GPUs.
  - **SIMD Extensions in CPUs:**
    - Intel's **AVX (Advanced Vector Extensions)** or ARM's **NEON**.
- **Use Case:**
  - Applications like matrix multiplication, image processing, machine learning, or any task with large-scale repetitive operations.



2. Single Instruction, Multiple Data (SIMD)



CU= Control Unit
PE= Processing Element
M= Memory
IS= Instruction Stream
DS= Data Stream
LM=Local Memory

2. Single Instruction, Multiple Data (SIMD)

PEs – Array Processing Elements:

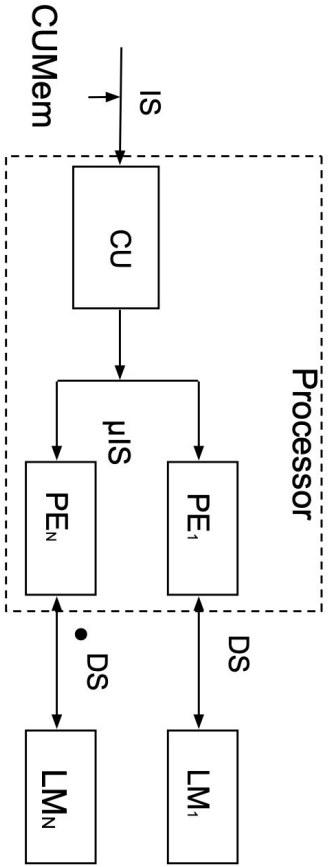
- SIMD systems consist of multiple **processing elements (PEs)** that work in parallel.
- Each PE is responsible for processing a portion of the data array simultaneously.
- Example: In a GPU, "cores" are the processing elements that operate on multiple pixels or data chunks in parallel.

Hardware Synchronization by the Same Processor:

- All PEs are synchronized by a **single control unit or processor**.
- The same instruction is broadcast to all PEs, and they execute it simultaneously on different data elements.
- This synchronization ensures uniformity and efficiency in SIMD operations.

Exploits Spatial or Data Parallelism:

- SIMD excels at **data parallelism**, where the same operation is applied to different pieces of data simultaneously.
- **Spatial parallelism** means the data being processed is stored in a way that makes it easily accessible to the PEs (e.g., stored contiguously in memory).
- Typical data structures: Arrays, matrices, and other contiguous data structures that facilitate parallel access.



3. Multiple Instructions, Single Data (MISD)

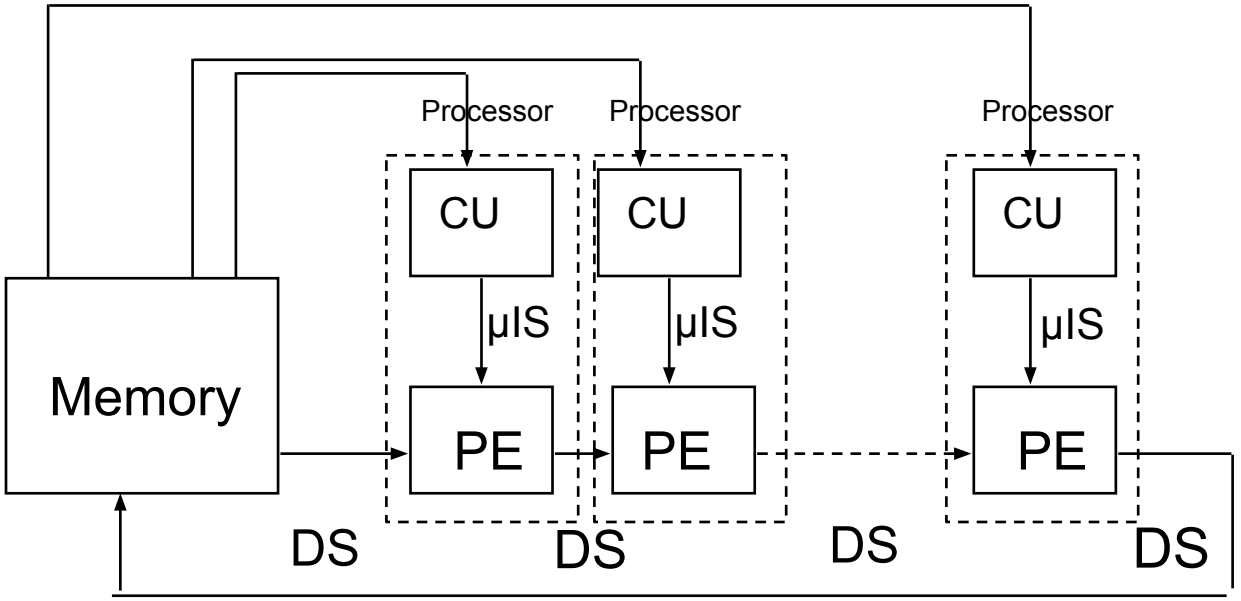
Definition:

- Multiple instructions operate on the same data stream.
- Rare in practice, mostly theoretical or used in specialized systems.

Examples:

- **Theoretical Example:**
  - Fault-tolerant systems where multiple processors execute the same operations redundantly to detect errors
  - **Space Shuttle Flight Control Computers:** Ensure reliability by comparing outputs of redundant operations
- **Modern Example:**
  - **Some systolic arrays** or specialized pipelines for error detection and control (e.g., certain control systems in spacecraft or avionics).
- **Use Case:**
  - Fault tolerance, niche control systems.
  - Multiple processors execute **different instructions** on the same data.
  - Outputs are compared to detect and mask errors (e.g., **redundancy for reliability**).

3. Multiple Instructions, Single Data (MISD)



CU= Control Unit,
PE= Processing Element,
M= Memory

4. Multiple Instructions, Multiple Data (MIMD)

Definition:

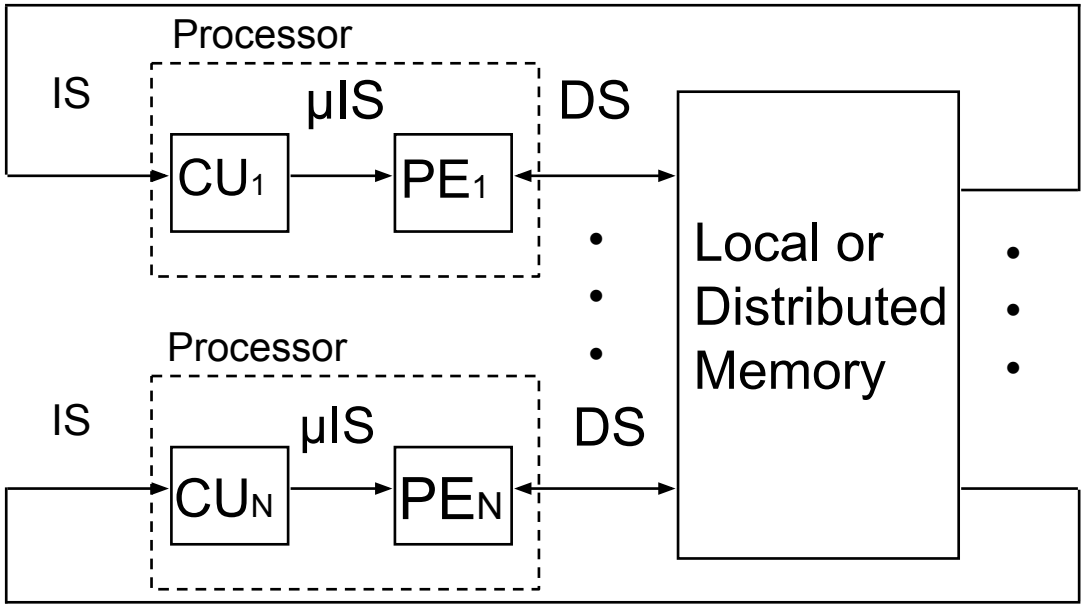
- Multiple processors execute independent instructions on independent data streams.
- The most common category for modern parallel computing.

Examples:

- **Classical Example:**
  - Early distributed systems like **IBM SP-2** or **Beowulf clusters**.
- **Modern Example:**
  - **Multi-core Processors:**
    - Intel Core i9 or AMD Ryzen (each core runs independent threads).
  - **Distributed Systems:**
    - Cloud computing (e.g., AWS clusters, Google TPU pods).
  - **Supercomputers:**
    - Modern supercomputers like the **Fugaku** or **Summit** (MIMD nodes communicate via networks).
- **Use Case:**
  - High-performance computing (HPC), distributed databases, or real-time applications.

https://www.hpe.com/emea_europe/en/compute/hpc/lawrence-livermore-national-laboratory.html

4. Multiple Instructions, Multiple Data (MIMD)



CU= Control Unit
PE= Processing Element
M= Memory
IS= Instruction Stream
DS= Data Stream

Flynn Classification

Category	Definition	Classical Example	Modern Example	Use Case
SISD	Single instruction, single data	Intel 8086, ENIAC	Single-core CPUs	Basic sequential tasks
SIMD	Single instruction, multiple data	Cray-1	GPUs, AVX, TPUs	Vector operations, machine learning
MISD	Multiple instructions, single data	Fault-tolerant systems	Control systems, niche systolic arrays	Error detection, redundancy
MIMD	Multiple instructions, multiple data	IBM SP-2	Multi-core CPUs, cloud clusters	HPC, cloud computing, distributed systems

Systolic Arrays

Definition of Systolic Arrays:

- Arrays of processors arranged in a regular, grid-like network. Data flows rhythmically through the array, and each processor performs a small portion of the computation.
- Typically used for repetitive, highly parallel computations.

Classification:

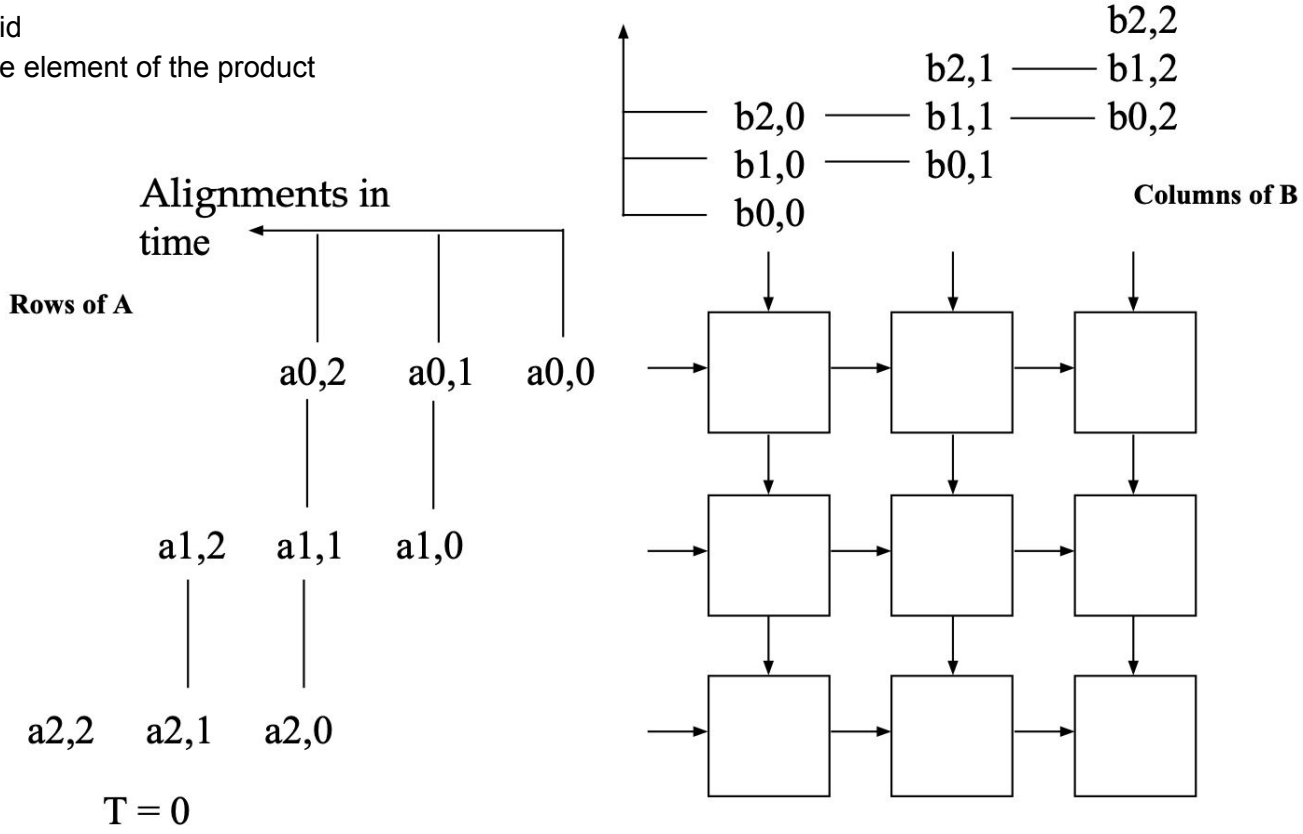
- **SIMD:**
  - Systolic arrays operate like SIMD systems when the same operation is performed on multiple data elements in a predictable pattern (e.g., matrix multiplication in machine learning).
- **MISD:**
  - In some configurations, systolic arrays could act as MISD systems when different processors perform distinct instructions on the same data stream, though this is less common.

Modern Examples of Systolic Arrays:

1. **TPUs (Tensor Processing Units):**
   - Google's TPUs use systolic arrays to perform matrix multiplications for machine learning tasks efficiently.
2. **FPGAs:**
   - Field-programmable gate arrays often implement systolic arrays for specialized applications like signal processing or AI inference.

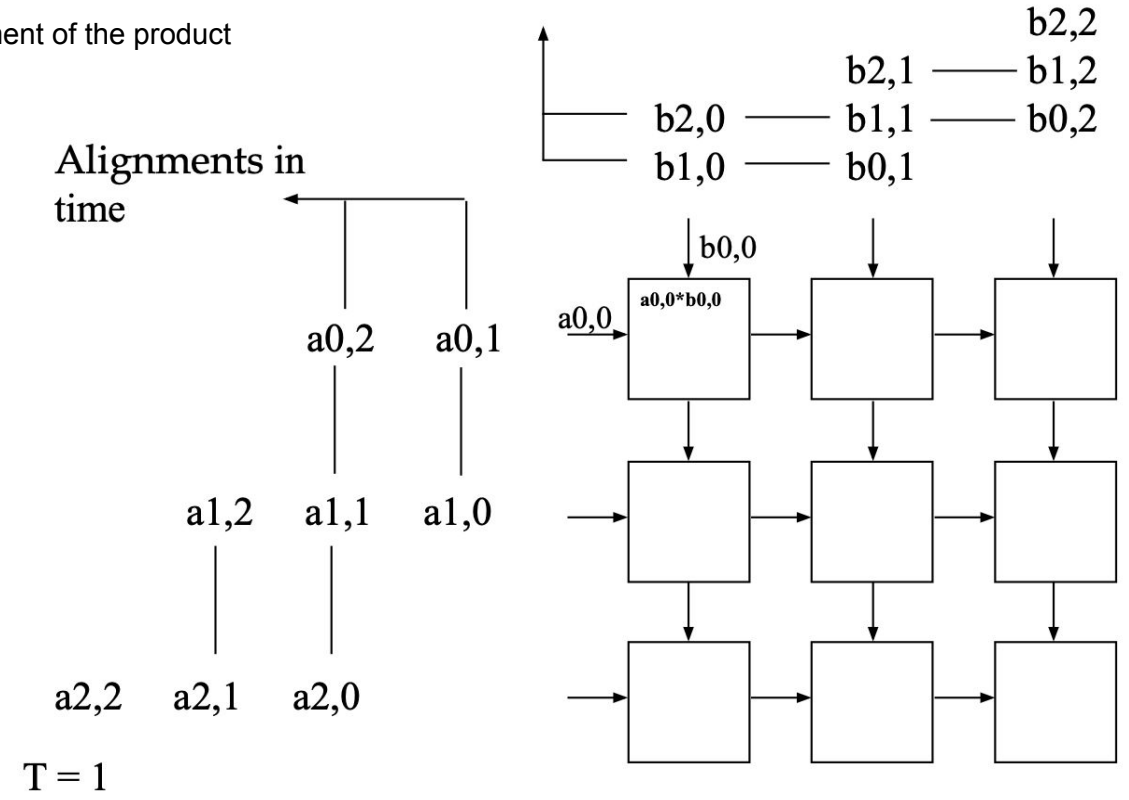
Systolic Array Example: 3x3 Systolic Array Matrix Multiplication

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product



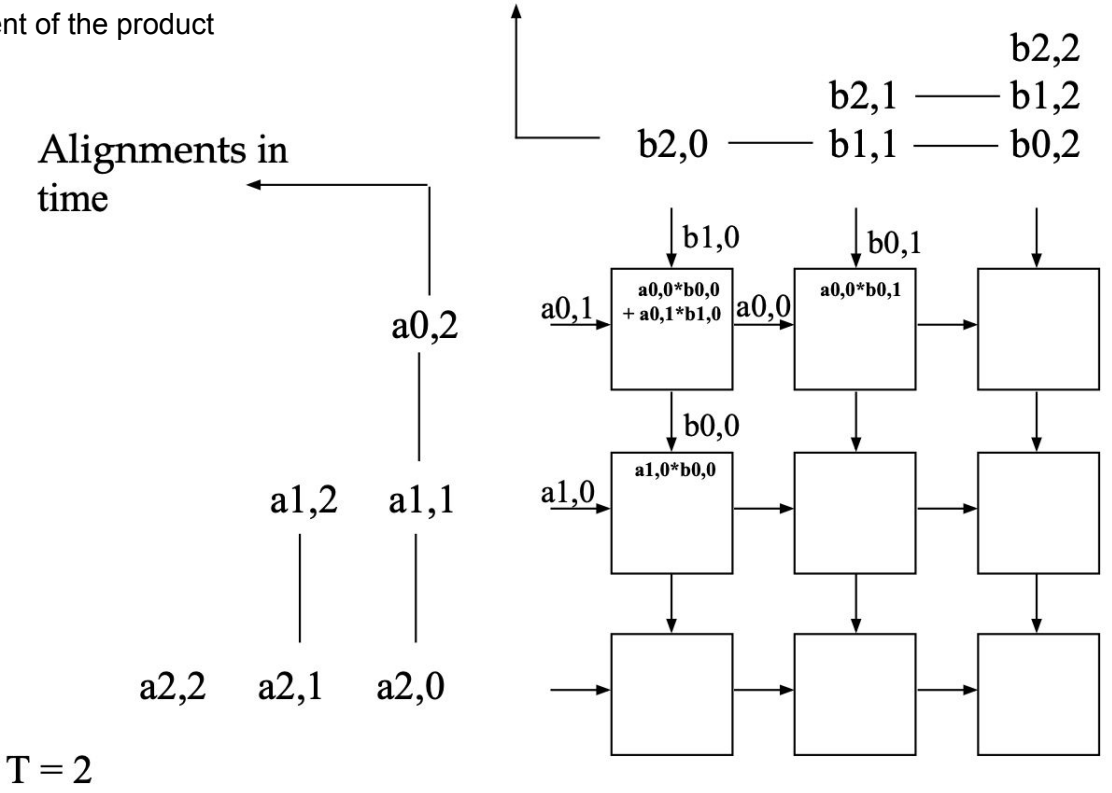
Systolic Array Example: 3x3 Systolic Array Matrix Multiplication

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product



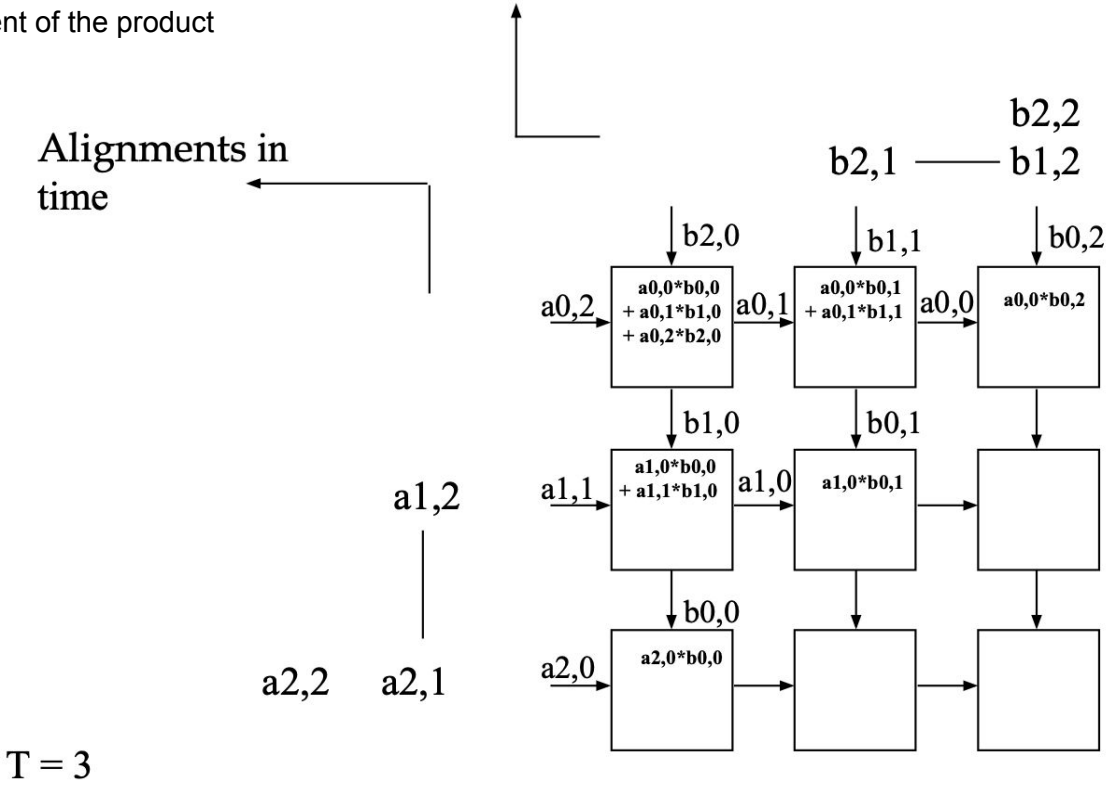
Systolic Array Example: 3x3 Systolic Array Matrix Multiplication

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product



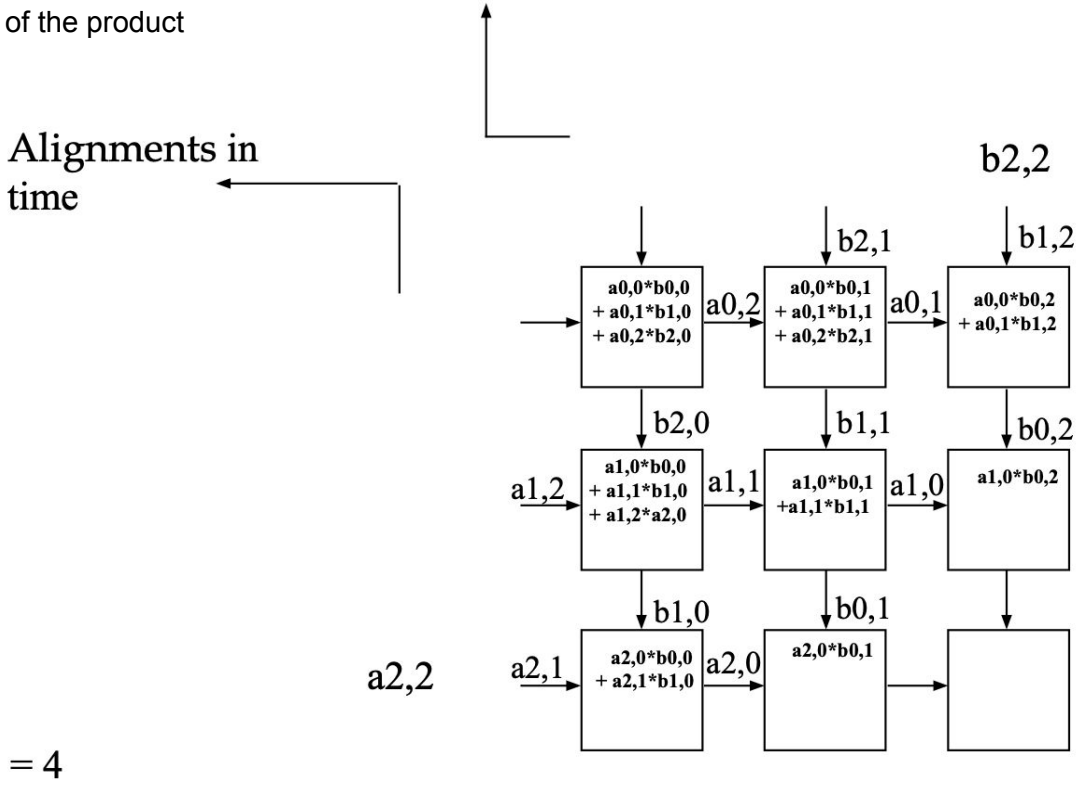
Systolic Array Example: 3x3 Systolic Array Matrix Multiplication

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product



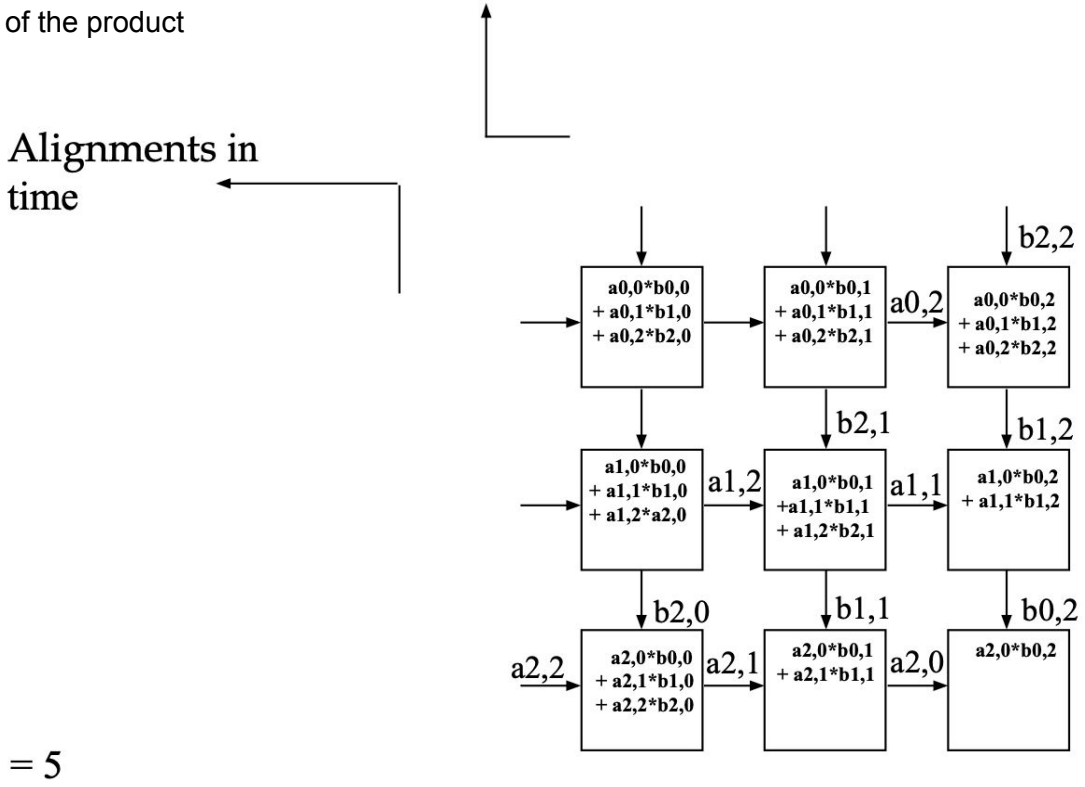
Systolic Array Example: 3x3 Systolic Array Matrix Multiplication

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product



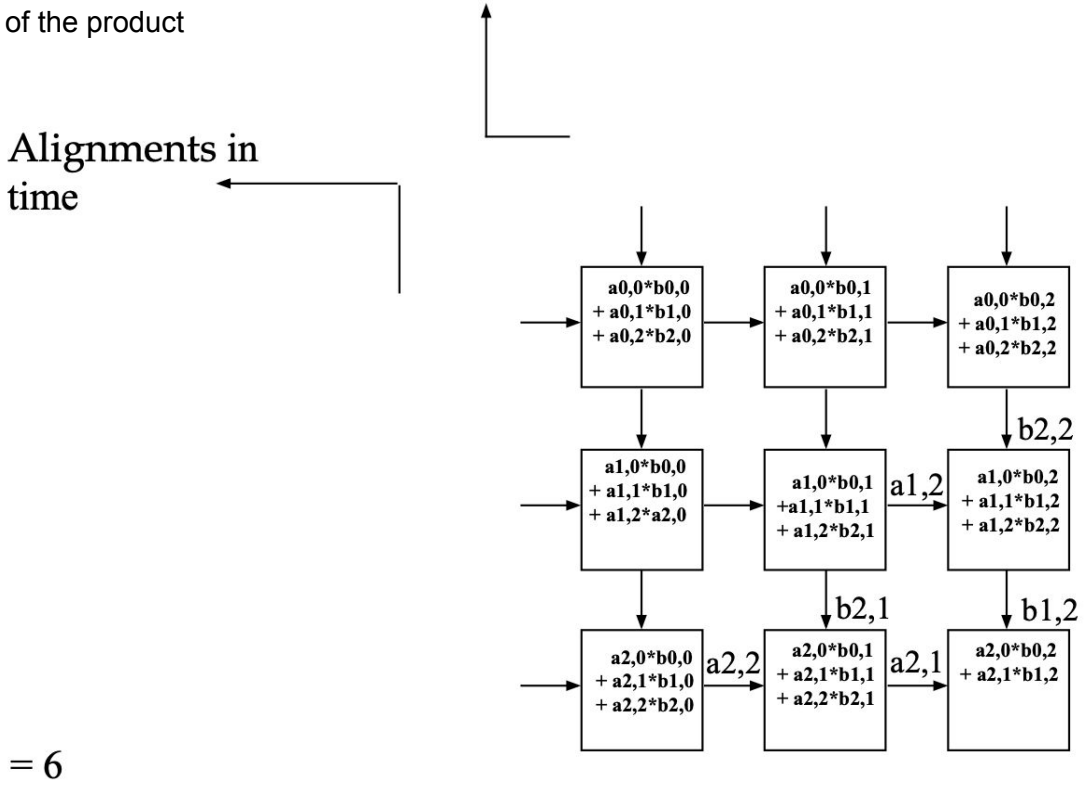
Systolic Array Example: 3x3 Systolic Array Matrix Multiplication

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product



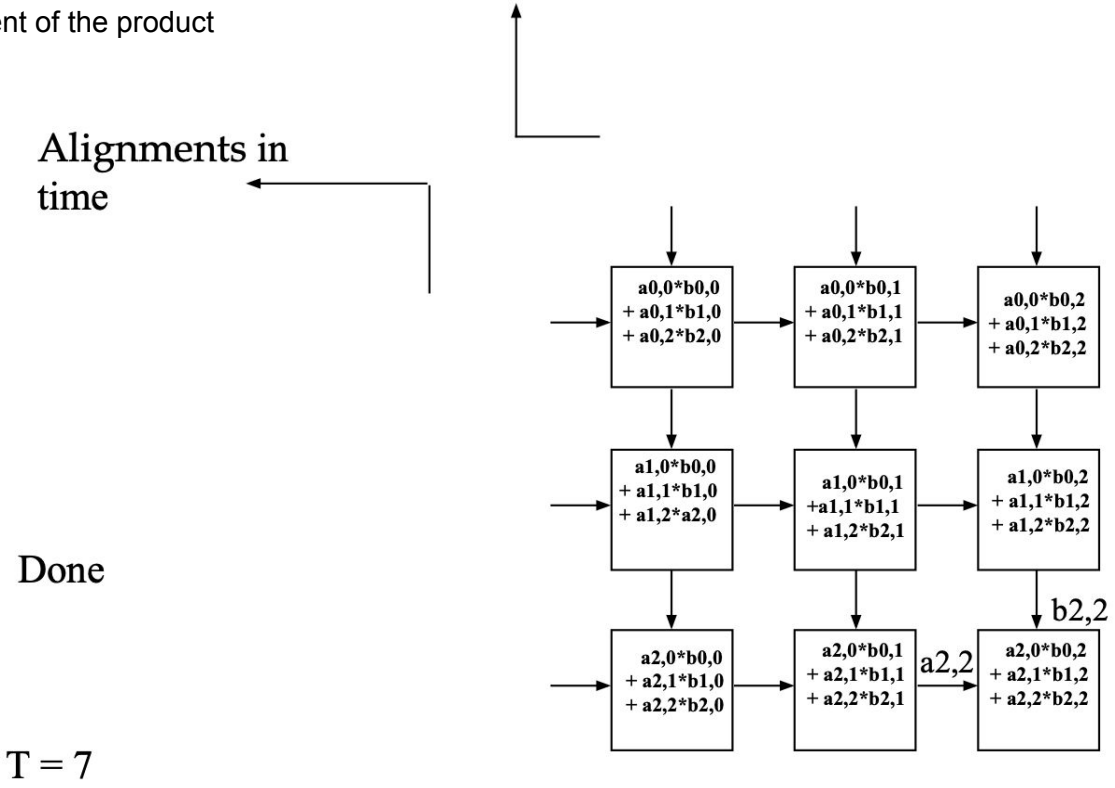
Systolic Array Example: 3x3 Systolic Array Matrix Multiplication

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product



Systolic Array Example: 3x3 Systolic Array Matrix Multiplication

- Processors arranged in a 2-D grid
- Each processor accumulates one element of the product



Parallelism Styles in Programs

Data Parallelism

- **Definition:** Many data elements processed in the **same manner** simultaneously.

- **Programming Examples:**
  - GPU programming with **CUDA/OpenCL**: Applying a kernel to every element in a matrix.
  - High-level frameworks:
    - **TensorFlow** for neural networks.
    - **NumPy's** `vectorize` or parallel loops for mathematical operations on arrays.
  - Real-world example:
    - **Image Processing:** Applying a filter (e.g., Gaussian blur) to every pixel of an image in parallel.

- **Hardware Connection:**
  - Efficiently implemented on **SIMD architectures** like GPUs, vector processors, or CPUs with SIMD extensions (e.g., Intel AVX).

Functional Parallelism

- **Definition:**Independent functions or program modules execute concurrently.
- **Programming Examples:**
  - Multi-threaded programs:
    - **Web Servers:** Threads handling multiple client requests independently (e.g., using Python's `threading` module or C++ `std::thread`).
    - **Data Pipelines:** ETL (Extract, Transform, Load) processes with independent stages running in parallel.
  - Distributed systems:
    - **MapReduce Frameworks (Hadoop):** Splitting data processing tasks across nodes.

Real-World Example:

- **Web Servers:** Separate threads handle tasks like request parsing, database queries, and response generation simultaneously for different parts of the application.

Hardware Connection:

- Best suited for **MIMD architectures**, such as multi-core CPUs, clusters, and distributed systems.

Functional Parallelism

- **Definition:** Tasks are arranged to overlap their execution, reducing idle time.
- **Programming Examples:**
- **Asynchronous Programming:**
  - Using Python's `asyncio` to overlap I/O-bound operations (e.g., fetching multiple webpages) or non-blocking I/O in C
- **Instruction Pipelines in CPUs:**
  - Fetch → Decode → Execute stages overlap for better throughput.

Real-World Example:

- **Network Packet Processing:**
  - While one packet is being sent, the next packet is prepared, reducing transmission delay.
- **Matrix Multiplication with Pipelining:**
  - A systolic array processes partial products of a matrix while new rows and columns are streamed.

Hardware Connection:

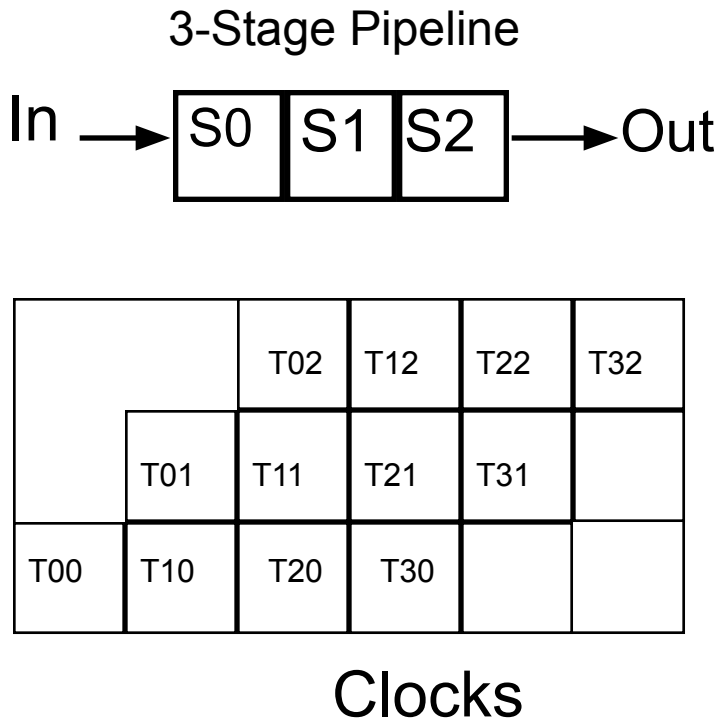
- Implemented in **pipelined architectures**, common in all modern CPUs and specialized hardware

Pipelining

- Tasks are divided into sequential stages, with each stage performing part of the work.
- **Goal:** Increase throughput by overlapping the execution of tasks.

Example – Floating Point Addition Pipeline:

1. **Stages:**
   - **Align:** Align operands to the same exponent.
   - **Add:** Perform the addition operation.
   - **Normalize:** Adjust the result to standard floating-point format.
2. **Scenario:**
   - 4 tasks, each requiring all 3 stages.
   - Pipeline takes **6 clocks** to complete all tasks, compared to **12 clocks** in a non-pipelined system.
   - **Performance:** Up to **3x faster** when the number of tasks is large.



Pipelining

Real-World Application – Instruction Pipelines:

- Example: Pipelined CPU processing where stages include:
  1. **Instruction Fetch (IF):** Retrieve instruction from memory.
  2. **Decode (ID):** Interpret the instruction.
  3. **Operand Fetch (OF):** Retrieve required data.
  4. **Execute (EX):** Perform the operation.
  5. **Write Back (WB):** Store results back into memory or registers.
- Improves throughput by overlapping these stages across multiple instructions.

Key Advantage:

- Exploits **overlapped/temporal parallelism** to maximize system efficiency.

Data Hazards

Data Hazards in Pipelines

- **Definition:** Occurs when an instruction depends on the result of a previous instruction that has not yet completed.

Types of Data Hazards:

1. **RAW (Read After Write):**

Example:
assembly
CopyEdit

```
ADD R1, R2, R3  // Produces result in R1
SUB R4, R1, R5  // Needs R1 but ADD is not finished yet
```

  - Dependency stalls the pipeline.

Resolution Techniques:

- **Forwarding/Bypassing:** Use intermediate results directly from pipeline stages.
- **Stalling:** Delay dependent instructions until the hazard resolves.

Structural Hazards

- **Definition:** Occurs when multiple instructions require the same hardware resource at the same time.

Examples:

1. **Single Memory Port:**
   - One instruction is fetching data from memory while another needs to write results back to memory.
2. **Shared ALU:**
   - Multiple floating-point instructions require the ALU simultaneously.

Impact:

- Prevents the pipeline from executing instructions in parallel.
- Introduces stalls until the resource becomes available.

Resolution Techniques:

- **Increase Resources:** Add duplicate units (e.g., dual-port memory or multiple ALUs).
- **Stalls:** Serialize access when resources are limited.

Control Hazards

Occurs when the pipeline is unsure of which instruction to fetch next due to branches or jumps.

Example:

```
BEQ R1, R2, LABEL  // Conditional branch
ADD R3, R4, R5     // Next instruction depends on the branch outcome
```

- The pipeline must wait to determine whether the branch is taken or not.

Resolution Techniques:

1. **Branch Prediction:**
   - Predict the branch outcome to keep the pipeline moving.
   - Modern CPUs achieve ~90-95% prediction accuracy.
2. **Speculative Execution:**
   - Execute instructions along the predicted path but discard them if the prediction is wrong.
3. **Pipeline Flush:**
   - Clear incorrect instructions when a branch misprediction occurs, introducing a stall.

Impact:

- Reduces pipeline efficiency, especially in branch-heavy programs.

Operational Models for Parallel Computers

Operational Models for Parallel Computers

Basic Categories of Commercial High-Performance Computers (Historical Perspective):

- **SIMD:** Now integrated into modern processors (e.g., GPUs, Intel AVX, ARM NEON).
- **MIMD:** Basis for multi-core CPUs and distributed systems.
- **Vector Processors:** Historical standalone systems; vectorization concepts live on in SIMD extensions.
- **Clusters:** Modern clusters are heterogeneous, combining CPUs, GPUs, and accelerators.

Modern Systems:

- Employ a mix of these models at different levels of architecture.
    - Example: Supercomputers like Fugaku and Summit integrate SIMD, MIMD, and cluster architectures.

SIMD and Vector Processors

Operational Models for Parallel Computers: Vector Processors

Key Features:

1. **Efficient Vector Processing:**
   - Replace entire loops manipulating arrays with a single vector instruction.
2. **Memory Optimization:**
   - Fetch entire arrays from memory in a **pipelined** manner, leveraging **parallel high-speed interleaved memory**.
3. **Vector Registers:**
   - Store and operate on entire arrays in **CPU vector registers**, minimizing memory access delays.
4. **Support for Scalar Operations:**
   - Includes both scalar and vector functional units and instructions, supporting scalar and vector data.

Classes of Operations:

- **Scalar and Vector Operand Combinations:**
  - `S x V -> V`: Scalar times vector produces a vector (e.g., scaling a vector).
  - `V -> S`: Vector reduces to scalar (e.g., sum of vector elements).
  - `V x V -> V`: Vector times vector produces a vector (e.g., element-wise multiplication).
  - `V -> V`: Vector to vector operations (e.g., transformations like negation).

Decline of Standalone Vector Processors

Historical Dominance (1970s-1980s):

- Vector processors like the **Cray-1** and **NEC SX series** were highly successful in scientific and engineering applications, leveraging their ability to process arrays efficiently.

Why They Declined:

1. **Cost of Custom Hardware:**
   - Standalone vector processors required specialized, expensive hardware, which became less competitive as general-purpose processors improved.
2. **Scalability Issues:**
   - Vector processors excel at data-parallel workloads but struggled to adapt to task-parallel workloads and massively distributed systems (e.g., modern supercomputers).
3. **Rise of Commodity Hardware:**
   - The emergence of **commodity microprocessors** (MIMD systems with SIMD extensions) offered cost-effective alternatives that combined vector-like capabilities with flexibility.
4. **Shift to Clusters:**
   - Distributed systems and clusters using off-the-shelf processors became the dominant architecture for high-performance computing (HPC), reducing the demand for custom vector hardware.

Operational Models for Parallel Computers: **SIMD**

Key Features:

- **SIMD (Single Instruction, Multiple Data):**
  - Architecture designed for **data parallelism**, applying the same operation to multiple data items simultaneously.
- **Historical Context:**
  - **1980s:** Standalone SIMD machines like Connection Machine, Thinking Machines CM-2.
  - **Today:** Integrated into modern CPUs, GPUs, and accelerators.

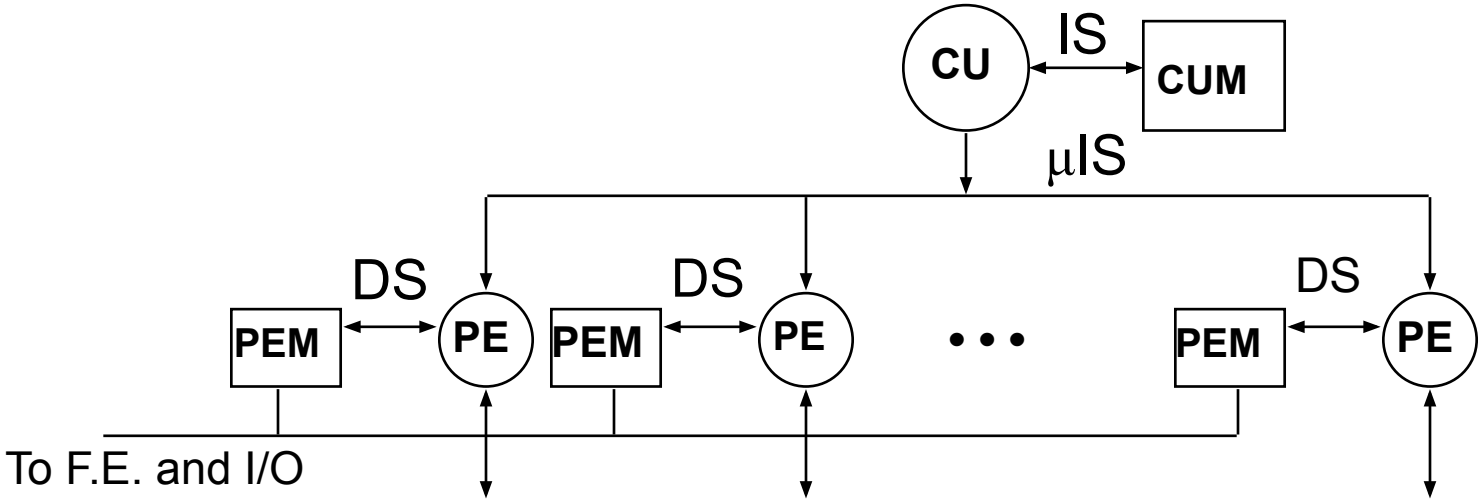
Modern SIMD Extensions:

- **CPU SIMD Extensions:**
  - Intel **AVX** (Advanced Vector Extensions), IBM VMX/Altivec, ARM NEON.
- **GPU SIMD-like Model:**
  - NVIDIA **SIMT** (Single Instruction, Multiple Threads).

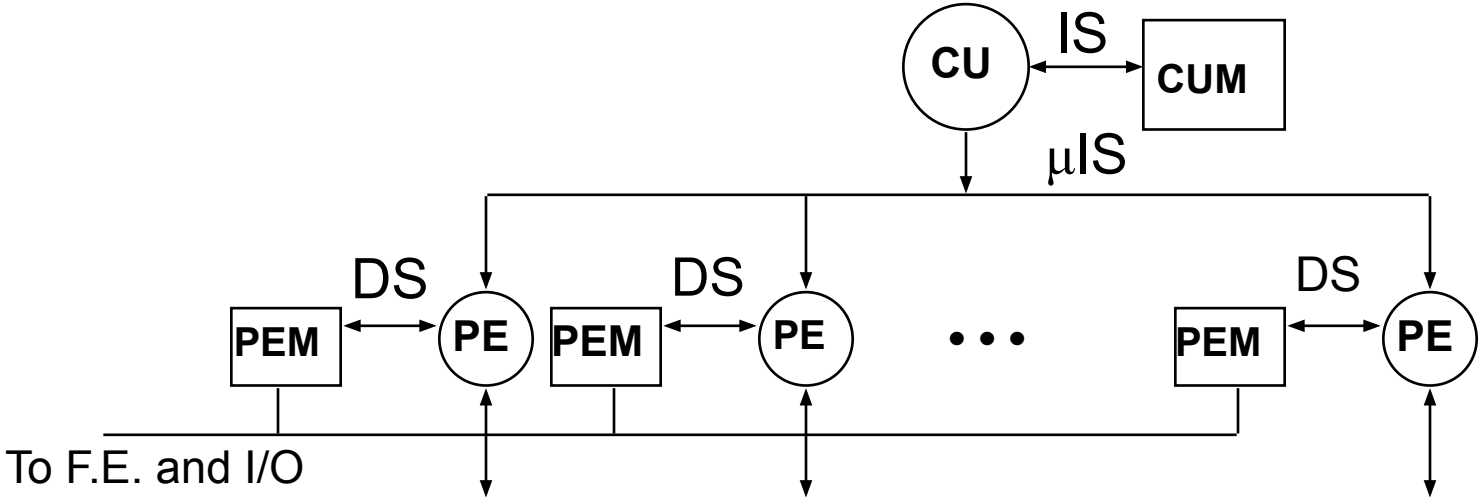
Technical Features:

- **Instruction Broadcast:** Microinstructions are broadcast to all processing elements.
- **Hardware Synchronization:** Ensures simultaneous operation on all elements.
- **Challenge – Independent Branching:**
  - Divergence occurs if different elements need to execute different instructions (e.g., in GPUs).

Operational Models for Parallel Computers: **SIMD**



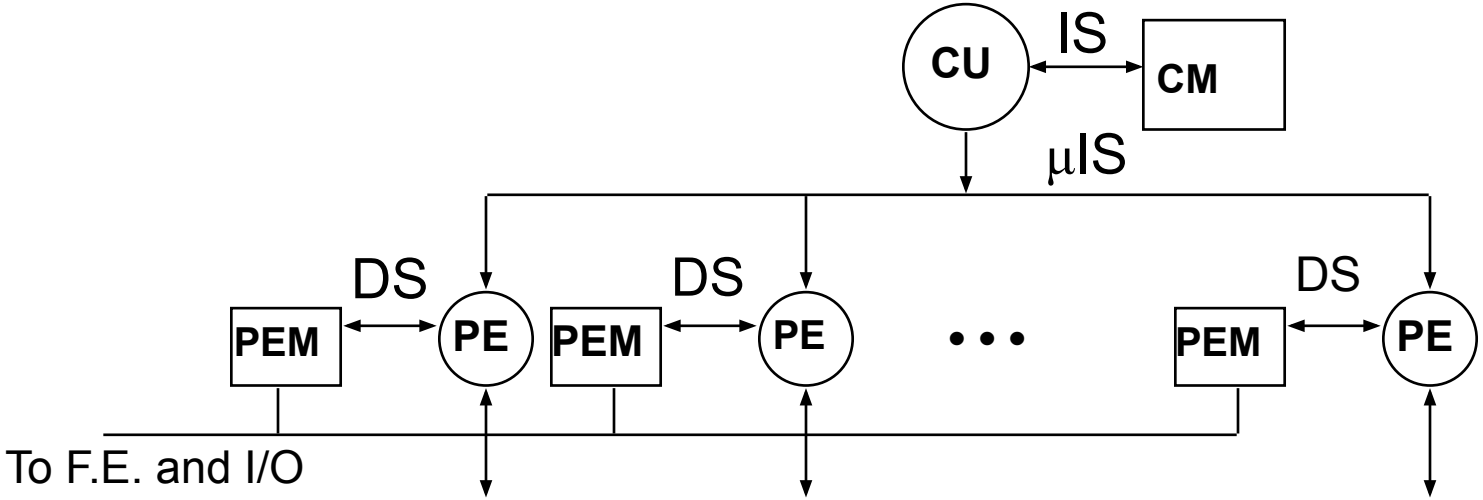
Operational Models for Parallel Computers: SIMD



CU (Control Unit):

- The **centralized controller** that issues **microinstructions (μIS)** to all the processing elements (PEs).
- Responsible for:
  - Decoding the instruction sequence (`IS` input).
  - Broadcasting instructions to all PEs.
- Ensures that all PEs execute the same operation synchronously.

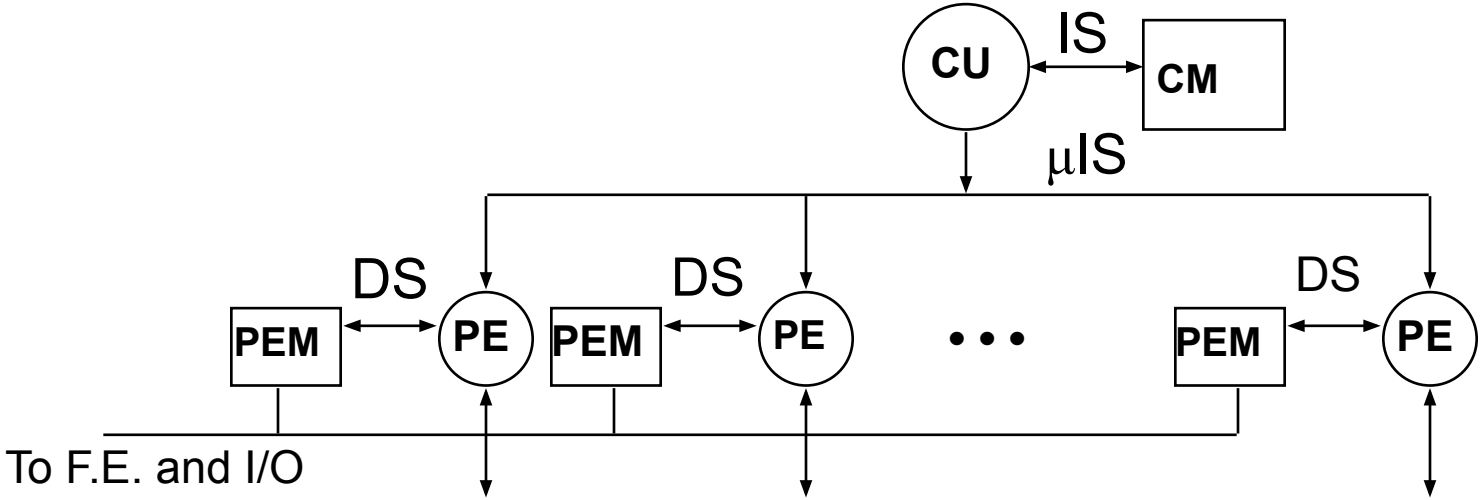
Operational Models for Parallel Computers: SIMD



CM (Control Memory):

- Stores the program or instructions (`IS`) that the control unit fetches and executes.
- Acts as the **program memory** for the SIMD system.

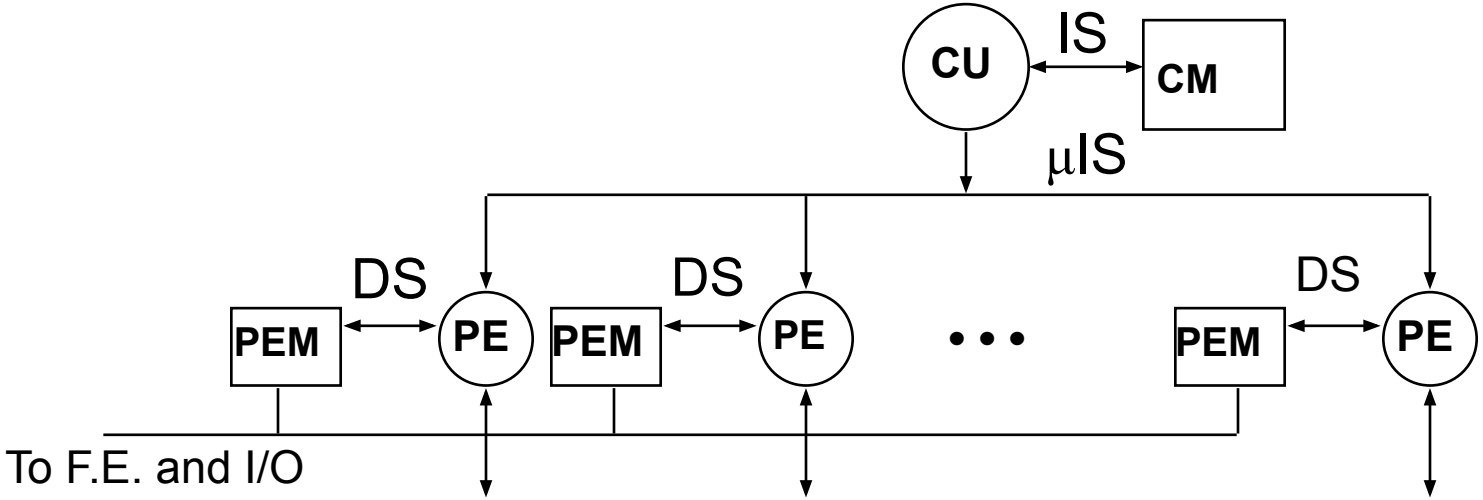
Operational Models for Parallel Computers: SIMD



PE (Processing Element):

- These are the **data processors** in the SIMD system.
- Each PE has:
  - **ALU (Arithmetic Logic Unit):** Performs arithmetic and logical operations.
  - **Local memory (PEM):** Holds the data assigned to that particular PE.
- PEs work independently on different pieces of data but execute the same instruction at the same time (as directed by the CU).

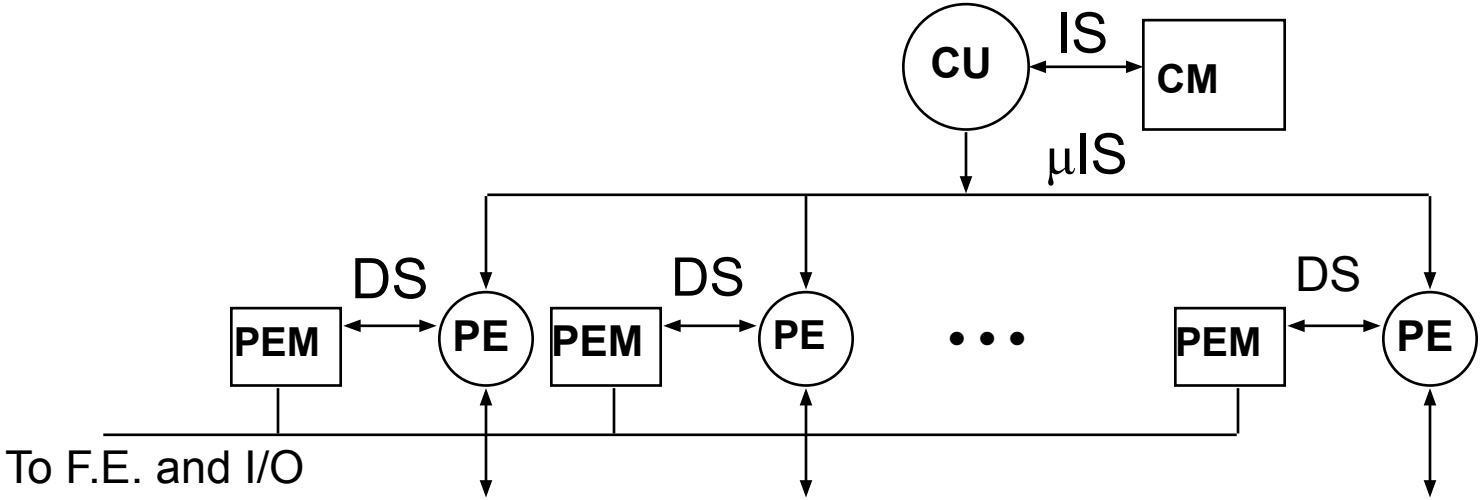
Operational Models for Parallel Computers: SIMD



PEM (Processing Element Memory):

- Local memory attached to each PE that stores the subset of data being processed.
- Enables distributed data storage for the parallel computation.

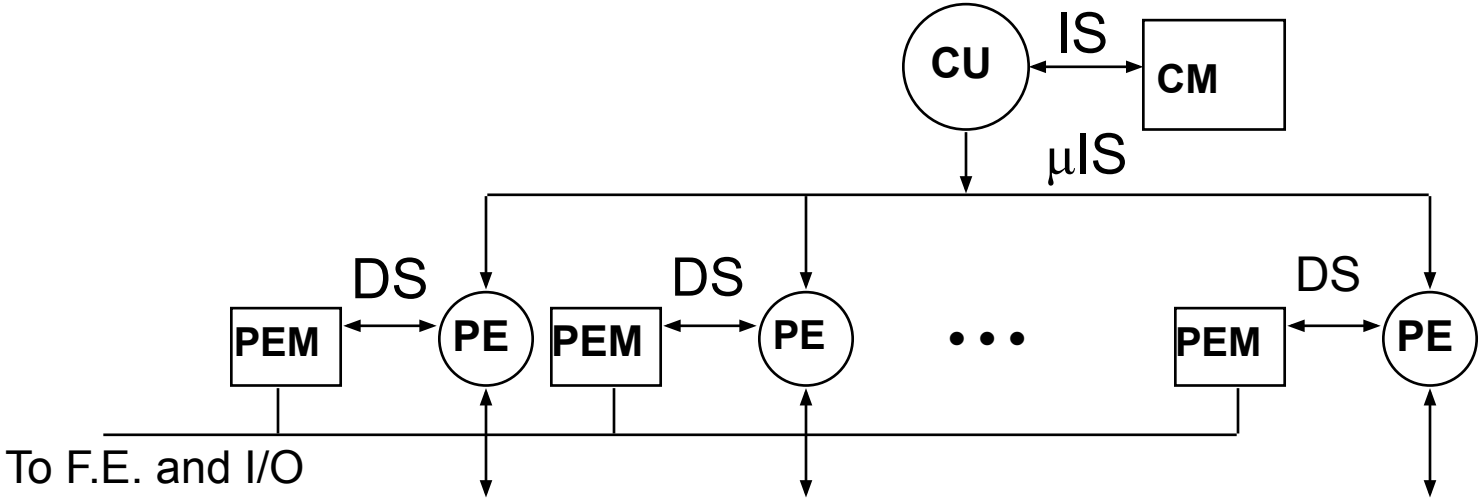
Operational Models for Parallel Computers: SIMD



DS (Data Stream):

- Represents **communication channels** between PEs, often enabling **nearest-neighbor communication.**
- For example, in operations like matrix averaging, data flows horizontally or vertically between PEs to exchange neighboring values.

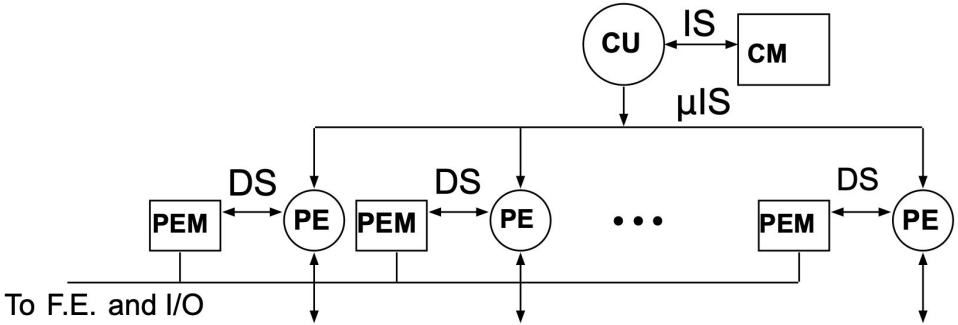
Operational Models for Parallel Computers: SIMD



To F.E. and I/O:

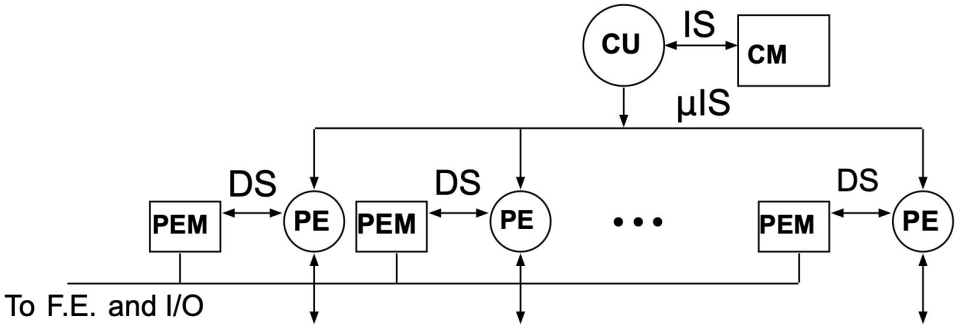
- Represents the interface for **front-end control and input/output** operations, which handle data input and results output for the SIMD system.

How it operates



1. **Instruction Fetch and Broadcast:**
   - The **control unit (CU)** fetches instructions from its memory (CUM) and broadcasts them as microinstructions (`μIS`) to all PEs.
2. **Parallel Data Processing:**
   - Each **processing element (PE)** performs the same operation on its local data (stored in PEM).
   - Example: If the operation is `ADD`, each PE adds its local value to a neighbor's value.
3. **Data Communication Between PEs:**
   - Data can flow between PEs through the **data stream (DS)** connections, enabling operations like matrix shifts, averaging, or stencil computations.
4. **Synchronization:**
   - All PEs operate in lockstep, meaning they execute the same instruction simultaneously, ensuring efficient parallelism.

How it operates



Example Usage: Matrix Averaging

To compute the average of a matrix element with its four neighbors:

1. Each PE is assigned one element of the matrix, stored in its **PEM (local memory)**.
2. The control unit broadcasts an instruction to **shift** data across neighbors using the **DS (data stream)** connections (e.g., shift left, right, up, down).
3. Each PE accumulates the values of its neighbors and computes the average locally.
4. This process is repeated for all matrix elements in parallel.

MIMD

Operational Models for Parallel Computers: **MIMD**

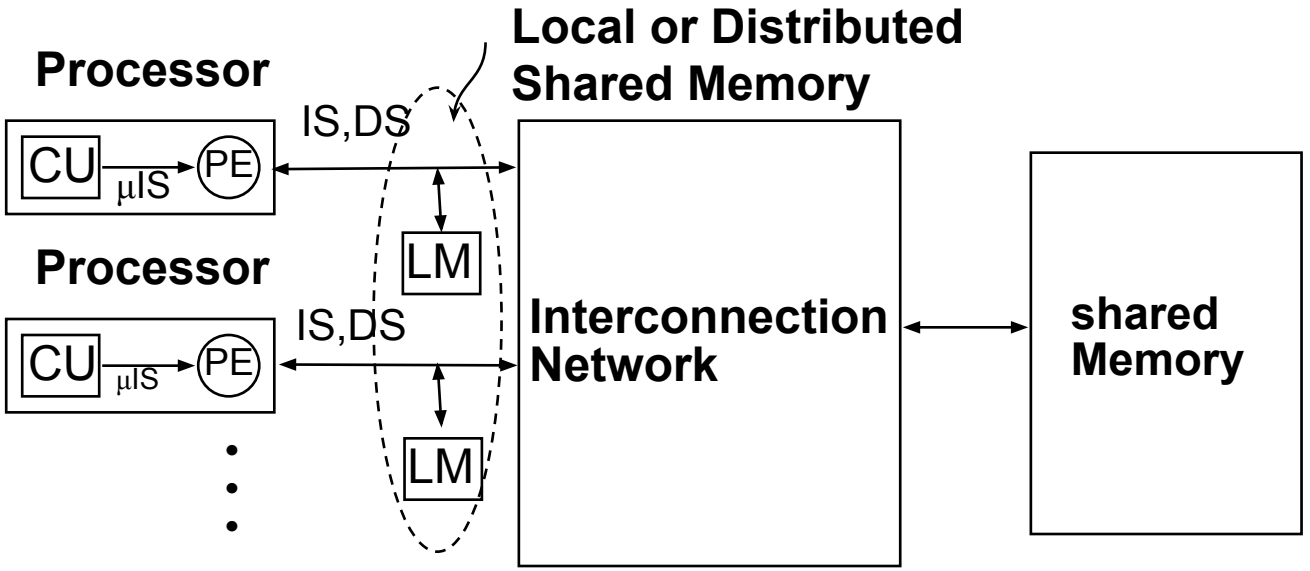
Key Features:

- **Control/Functional Parallelism:**
  - Emphasizes parallel execution of independent instructions across multiple processors.
- **Asynchronous Operation:**
  - Processors execute independently and do not require global synchronization.
- **Synchronization:**
  - Typically handled in software (e.g., via operating system or application-level tools).
  - Can emulate SIMD behavior using **SPMD (Single Program, Multiple Data)** for application-level synchronization.

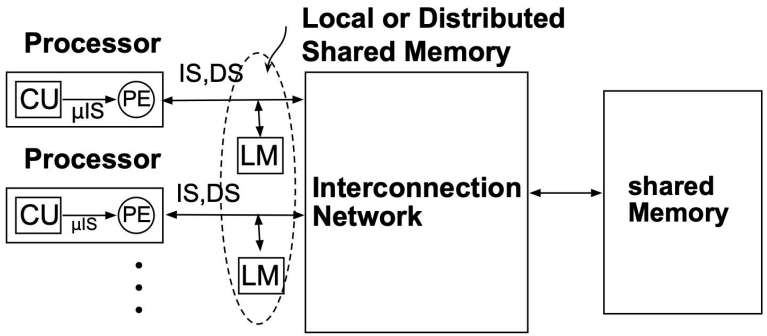
Examples:

- **Clusters and Supercomputers:**
  - IBM **BlueGene**, SGI **Altix**, Cray **XC50**, Sunway **TaihuLight**, Fugaku.
- **Modern Distributed Systems:**
  - Cloud computing platforms, large-scale HPC clusters, and GPU-accelerated distributed systems.

Operational Models for Parallel Computers: **MIMD**



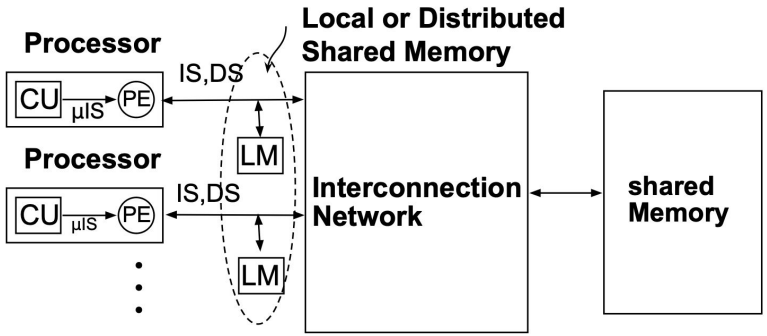
Operational Models for Parallel Computers: **MIMD**



Processor Units:

- Each processor contains:
  - **CU (Control Unit):**
    - Responsible for instruction sequencing and managing data flow within the processor.
  - **PE (Processing Element):**
    - Executes instructions and performs computations based on microinstructions (`μIS`) issued by the CU.
- **Role:** Each processor operates independently on its assigned tasks but may share data through the shared memory.

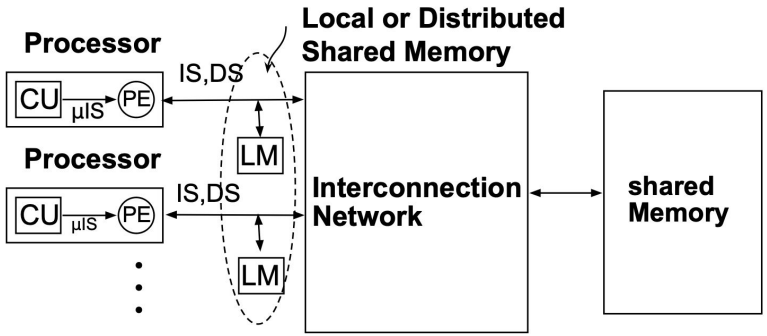
Operational Models for Parallel Computers: **MIMD**



Local Memory (LM):

- Each processor is connected to its **local memory**, which is private to the processor.
- **Purpose:**
  - Store frequently accessed data for faster processing.
  - Reduce contention for shared memory by holding temporary or local data.

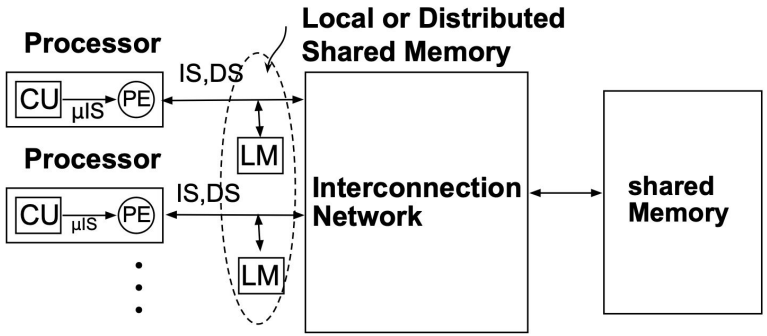
Operational Models for Parallel Computers: **MIMD**



Shared Memory:

- A **global memory** accessible by all processors through the interconnection network.
- **Purpose:**
  - Facilitates communication and data sharing between processors.
  - Example: Used for tasks like collaborative computation, where intermediate results are shared among processors.
  -

Operational Models for Parallel Computers: **MIMD**



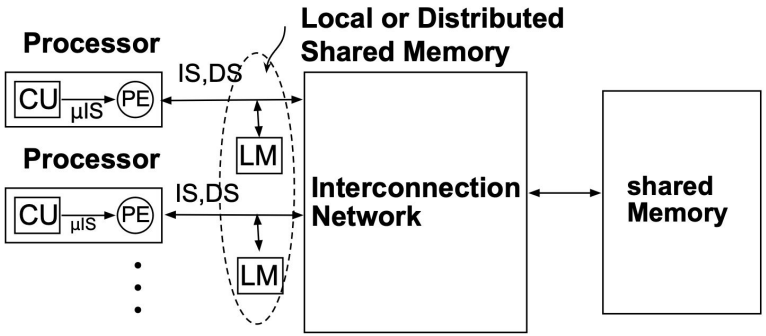
Interconnection Network:

- Acts as the bridge between processors (with their local memories) and the shared memory.
- **Types of Networks:**
  - **Bus-based:** Simple, but prone to bottlenecks.
  - **Switch-based (Crossbar or Mesh):** Scalable and allows simultaneous data transfers.
- **Role:** Handles communication between processors and shared memory, enabling data sharing or synchronization.

How it works

Data Access:

- Processors work on data stored in their **local memory (LM)** for fast, independent operation.
- If a processor needs shared data, it accesses **shared memory** through the interconnection network.



Communication:

- Shared memory is used for communication and synchronization between processors.
- For example, one processor writes results to shared memory, and another processor reads those results to continue computation.

Synchronization:

- The processors may need to synchronize to avoid conflicts or inconsistencies when accessing shared memory.
- Example: Locks, barriers, or semaphores are used to coordinate access.

Advantages and Challenges

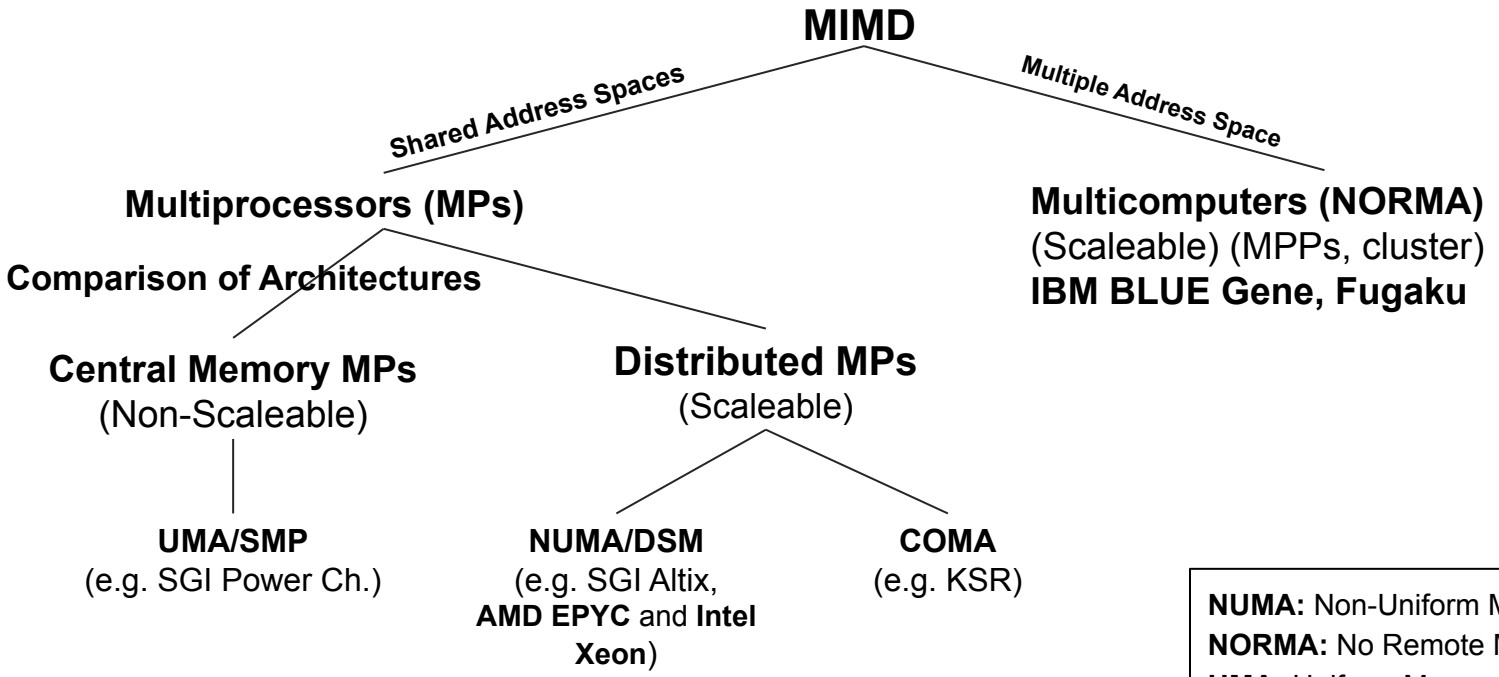
Advantages

1. **Ease of Programming:**
   - Shared memory provides a global address space, making it easier for programmers to manage data sharing.
2. **Fast Local Access:**
   - Processors can use local memory for frequently accessed data, reducing contention for shared memory.
3. **Scalability:**
   - With an appropriate interconnection network (e.g., crossbar or mesh), the architecture can scale to a large number of processors.

Challenges

1. **Memory Contention:**
   - Processors may compete for access to shared memory, causing delays.
2. **Synchronization Overhead:**
   - Ensuring consistency and preventing data races can add complexity and reduce performance.
3. **Interconnection Network Bottlenecks:**
   - The efficiency of the network affects overall system performance.

Taxonomy of MIMD



NUMA: Non-Uniform Memory Access
NORMA: No Remote Memory Access
UMA: Uniform Memory Access
COMA: Cache-Only Memory Access
SMP: Symmetric Multiprocessing
DSM: Distributed Shared Memory

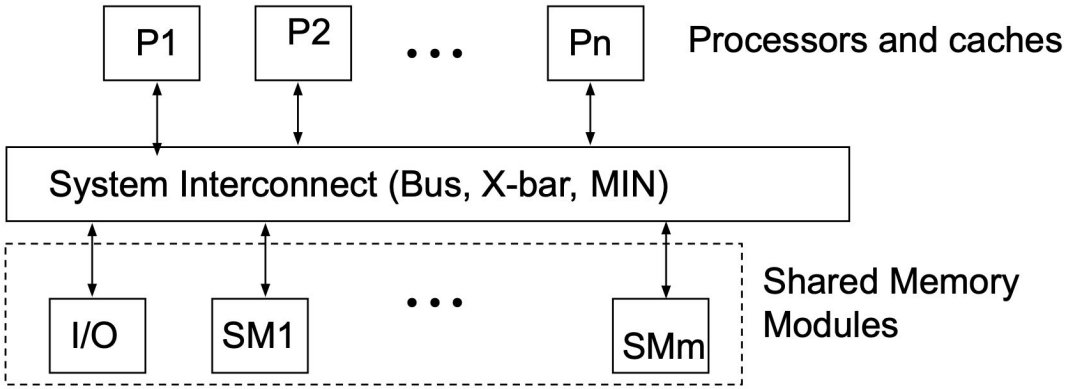
Note: a NUMA could be either cache-coherent or not

Uniform Memory Access (UMA) MP Model

Memory access time is **uniform** for all processors, regardless of which memory module is being accessed.

Symmetric Multiprocessing (SMP):

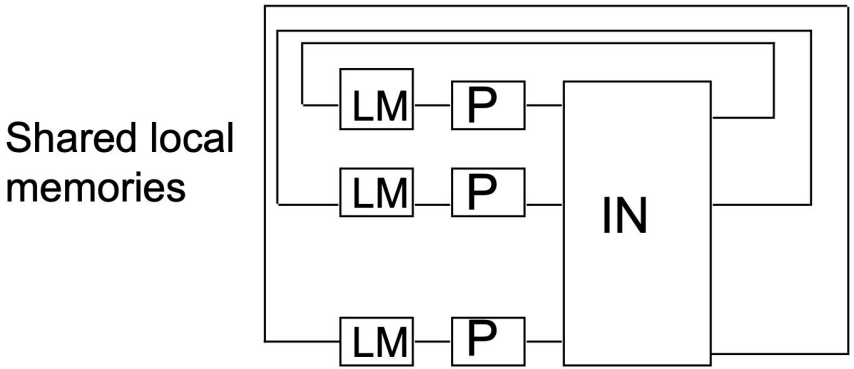
- All processors have **equal access** to:
  1. **Memory.**
  2. **Peripherals** (e.g., I/O devices).
  3. **Operating System Kernel.**
- Processors can execute **independent tasks** or work on the same task collaboratively.



Non-Uniform Memory Access (NUMA)

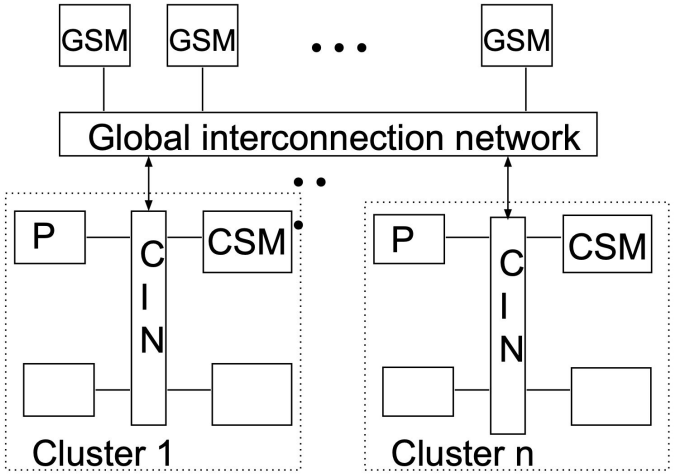
Key Features:

1. **Global Address Space:**
   - Local memory (LM) and global shared memory (GSM) are mapped into a single **global address space**accessible by all processors.
2. **Access Time Hierarchy:**
   - Fastest: **Local memory (LM)** or memory within the same cluster.
   - Second: **Global shared memory (GSM).**
   - Slowest: **Memory in an external cluster.**
3. **Access Time Variance:**
   - Memory access time depends on the location of the memory relative to the processor.
   - Designed to minimize latency by prioritizing access to nearbv memorv.



Hierarchical NUMA

- A memory architecture that organizes memory in a multi-level **hierarchical structure**, typically used in large-scale systems.
- Each **cluster** (or node) has its own local memory, but clusters are further grouped into higher levels of hierarchy.

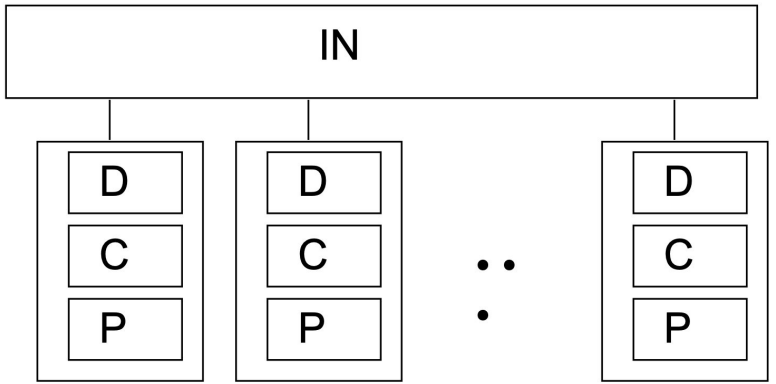


1. **Memory Access Levels:**
   - **Level 1:** Local memory (fastest access, lowest latency).
   - **Level 2:** Memory within the same cluster (CSM – Cluster Shared Memory).
   - **Level 3:** Global shared memory (GSM) in other clusters, accessed through the interconnection network (slower).
2. **Cluster Organization:**
   - Processors are grouped into **clusters**.
   - Clusters may be connected through a **Cluster Interconnection Network (CIN)** such as NUMAlink, InfiniBand, or Ethernet.
3. **Global Address Space:**
   - Memory from all levels is mapped into a single **global address space**, but the time to access memory depends on its distance from the processor.

Cache-Only Memory Access (COMA) Multiprocessors

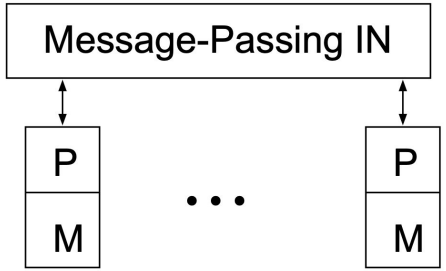
- A **special case of NUMA**, where all local memories are structured as **caches** (COMA caches).
- Unlike NUMA, COMA does not have fixed "home nodes" for memory addresses. Instead, data dynamically **migrates** to the processor(s) that access it most frequently.

1. **COMA Caches:**
   - All local memories are treated as large caches (e.g., KSR's "ALLCACHE" architecture).
   - COMA caches are much larger than typical L2 caches, making the system **expensive**.
2. **Dynamic Data Migration and Replication:**
   - Data is **migrated** and **replicated** dynamically based on access patterns, enabling efficient use of memory resources.
   - Memory coherence is maintained using **hardware mechanisms**.
3. **Elimination of Home Nodes:**
   - Unlike NUMA, there are no fixed "home nodes" for memory address
   - Data moves to where it is needed, reducing redundant copies.
4. **Challenges:**
   - **Data Location:** Without home nodes, locating specific data requires complex hardware mechanisms.
   - **Memory Pressure:** When local memory fills up, evictions are required. Evicted data has no home, complicating management.



Multicomputers – MPPs or Clusters

1. **Distributed Memory:**
   - Each processor has its **own private memory**.
   - No shared memory; processors communicate explicitly.
2. **No Remote Memory Access (NORMA):**
   - Processors cannot directly access the memory of other nodes.
   - Communication relies entirely on message-passing.
3. **Communication:**
   - **Message-Passing Interface (MPI):**
     - Standardized communication protocol for exchanging data between nodes.
   - **Network Interconnects:**
     - High-speed interconnects like InfiniBand or Cray Aries link the processors.

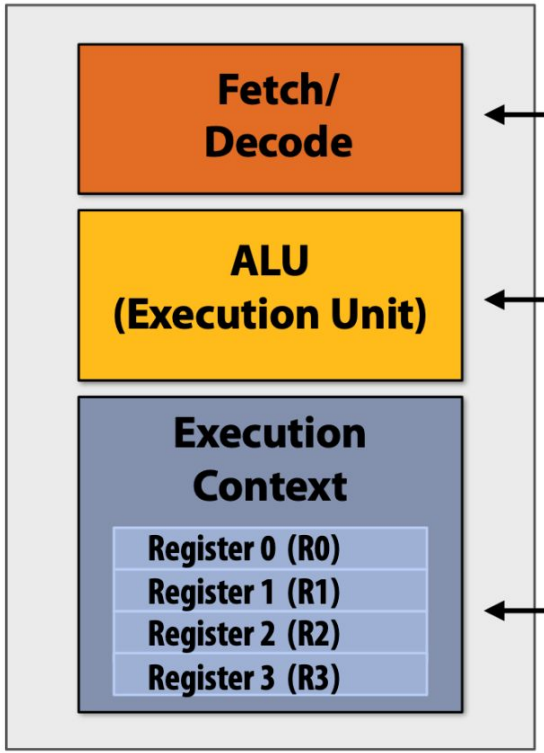


Challenges:

- **Programming Complexity:**
  - Requires explicit message-passing, increasing the burden on programmers.
- **Latency:**
  - Remote communication via interconnects is slower than accessing shared memory.

Intuitions for Parallelism

Parallelism Intuitions



Determine what instruction to run next

Execution unit: performs the operation described by an instruction, which may modify values in the processor's registers or the computer's memory

Registers: maintain program state: store value of variables used as inputs and outputs to operations

A Programs Perspective of Memory

A computer's memory is structured as an array of bytes.

- Each byte is assigned a unique **address**, which represents its position in the memory array.

(We'll continue assuming the memory is byte-addressable.)

- **Examples:**
  - The byte at address **0x2** contains the value **1**.
  - The byte at address **0x1F (31)** contains the value **255**.

Memory

Address	Value
0x0	10
0x1	0
0x2	1
…	
0x1F	255

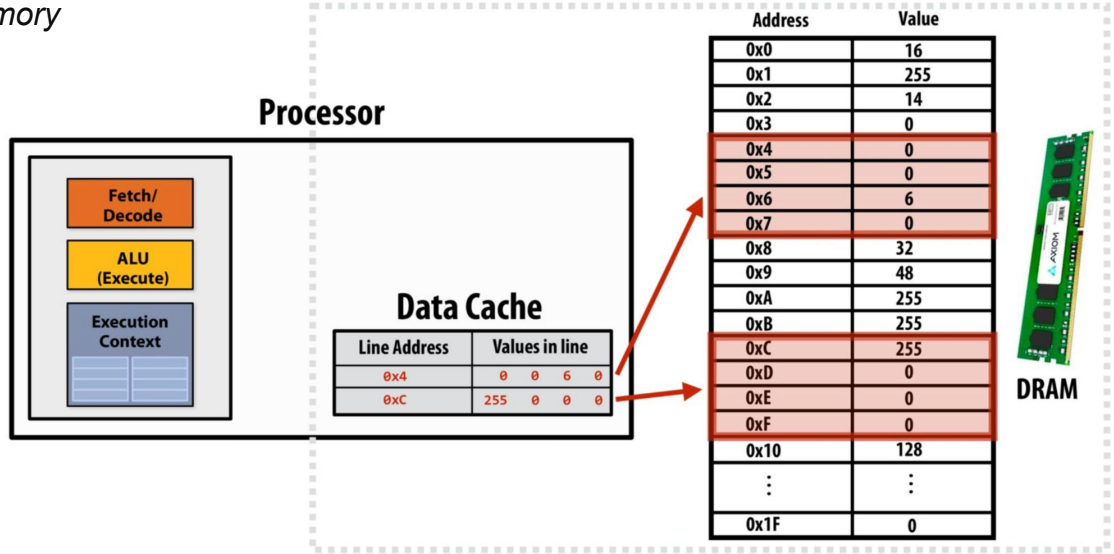
Cache

What Are Caches?

A cache is a small, high-speed hardware storage layer that holds frequently accessed data to reduce memory access latency.

Key Properties of Caches

- **Transparent to Software**
  - Does not alter program behavior.
  - Only impacts performance.
- **On-Chip Storage**
  - Stores a subset of data from main memory.
  - Faster access than DRAM (e.g., 1-5 cycles vs. 100+ cycles).
- **Cache Line Granularity**
  - Data stored in fixed-size blocks (*cache lines*).
  - Example: 4-byte line size.
- **Two ways caches help**
  - Spatial and Temporal locality



Cache example 1

Array of 16 bytes in memory

	Address	Value
Line 0x0	0x0	16
	0x1	255
	0x2	14
	0x3	0
Line 0x4	0x4	0
	0x5	0
	0x6	6
	0x7	0
Line 0x8	0x8	32
	0x9	48
	0xA	255
	0xB	255
Line 0xC	0xC	255
	0xD	0
	0xE	0
	0xF	0

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines
(So 2 lines fit in cache)

Least recently used (LRU)
replacement policy

time

Address accessed	Cache action	Cache state (after load is complete)
0x0	"cold miss", load 0x0	0x0 ●●●●
0x1	hit	0x0 ●●●●
0x2	hit	0x0 ●●●●
0x3	hit	0x0 ●●●●
0x2	hit	0x0 ●●●●
0x1	hit	0x0 ●●●●
0x4	"cold miss", load 0x4	0x0 ●●●● 0x4 ●●●●
0x1	hit	0x0 ●●●● 0x4 ●●●●

There are two forms of "data locality" in this sequence:

Spatial locality: loading data in a cache line "preloads" the data needed for subsequent accesses to different addresses in the same line, leading to cache hits

Temporal locality: repeated accesses to the same address result in hits.

Cache example 2

Array of 16 bytes in memory

	Address	Value
Line 0x0	0x0	16
	0x1	255
	0x2	14
	0x3	0
Line 0x4	0x4	0
	0x5	0
	0x6	6
	0x7	0
Line 0x8	0x8	32
	0x9	48
	0xA	255
	0xB	255
Line 0xC	0xC	255
	0xD	0
	0xE	0
	0xF	0

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines
(So 2 lines fit in cache)

Least recently used (LRU)
replacement policy

time

Address accessed	Cache action	Cache state (after load is complete)
0x0	"cold miss", load 0x0	0x0
0x1	hit	0x0
0x2	hit	0x0
0x3	hit	0x0
0x4	"cold miss", load 0x4	0x0, 0x4
0x5	hit	0x0, 0x4
0x6	hit	0x0, 0x4
0x7	hit	0x0, 0x4
0x8	"cold miss", load 0x8 (evict 0x0)	0x8, 0x4
0x9	hit	0x8, 0x4
0xA	hit	0x8, 0x4
0xB	hit	0x8, 0x4
0xC	"cold miss", load 0xC (evict 0x4)	0x8, 0xC
0xD	hit	0x8, 0xC
0xE	hit	0x8, 0xC
0xF	hit	0x8, 0xC
0x0	"capacity miss", load 0x0 (evict 0x8)	0x0, 0xC

How Does a Processor Decide What Data to Keep in Cache?

Cache Mapping Policies *(Simplified Overview)*

1. **Direct-Mapped Cache**
   - Each memory block maps to **exactly one location** in the cache.
2. **Set-Associative Cache**
   - Each memory block maps to a **set** of cache locations (e.g., 4-way associative).
3. **Fully Associative Cache**
   - Data can be placed **anywhere** in the cache (rarely used).

1. Direct-Mapped Cache

Rule: Each memory address maps to **exactly one cache line**.
Example:

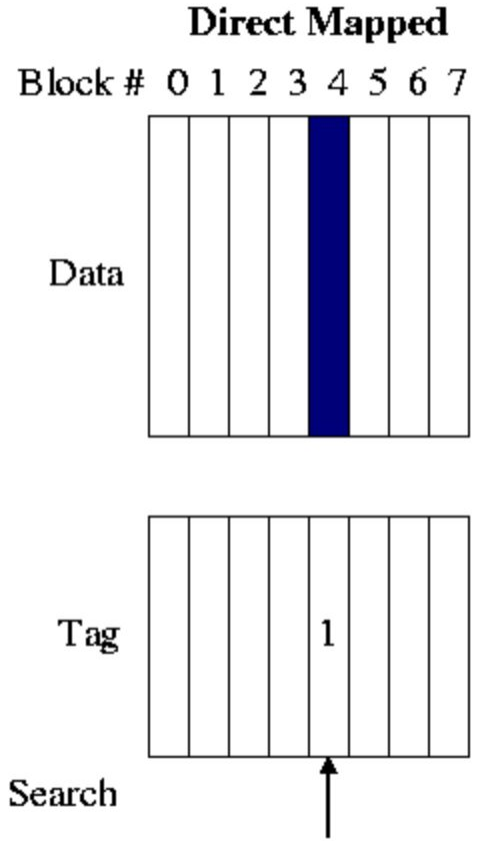
- **Cache Size:** 4 lines (0-3).
- **Memory Addresses:** `0x00`, `0x04`, `0x08`, `0x0C`.
- **Mapping Formula:**
  `Cache Line = (Address / Block Size) % Total Lines`.

Pros ✅

- **Simple hardware**: Easy to implement (no complex search logic).
- **Fast access time**: Fixed mapping means no decision-making overhead.
- **Low power consumption**: Minimal circuitry required.

Cons ❌

- **High conflict misses**: Multiple addresses compete for the same cache line.
- **Infexible replacement**: No choice in eviction—forced replacement on collision.
- **Poor utilization**: Frequently accessed blocks may evict each other.



2. Set-Associative Cache

Rule: Each memory address maps to a **set** (e.g., 2 lines per set).
Example:

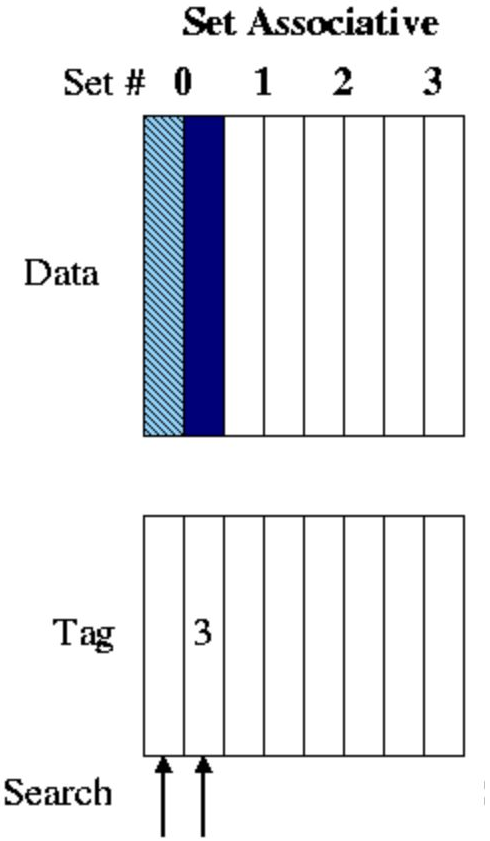
- **Cache:** 4 sets × 2 lines = 8 total lines.
- **Memory Addresses:** `0x00`, `0x08`, `0x10`.
- **Mapping Formula:** `Set = (Address / Block Size) % Total Sets`.

Pros ✅

- **Reduced conflict misses**: Multiple slots per set reduce collisions.
- **Balanced performance**: Better hit rate than direct-mapped, simpler than fully associative.
- **Flexible replacement**: Policies like LRU can be applied *within a set*.

Cons ❌

- **Increased complexity**: Additional logic to manage sets and replacement policies.
- **Slower than direct-mapped**: Slightly longer access time due to set search.
- **Higher power usage**: More circuitry for tag comparison.



3. Fully-Associative Cache

Rule: Any memory address can map to **any cache line**.
Example:

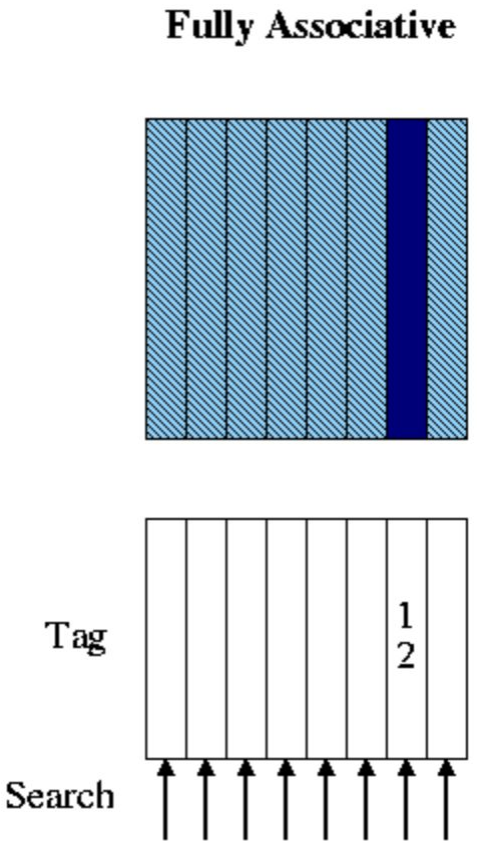
- **Cache Size:** 4 lines.
- **Memory Addresses:** `0x00`, `0x04`, `0x08`, `0x0C`, `0x10`.

Pros ✅

- **Minimal conflict misses**: Any block can occupy any cache line.
- **Optimal replacement**: Global LRU policy maximizes cache utilization.
- **Flexibility**: Ideal for small, high-priority caches (e.g., TLB).

Cons ❌

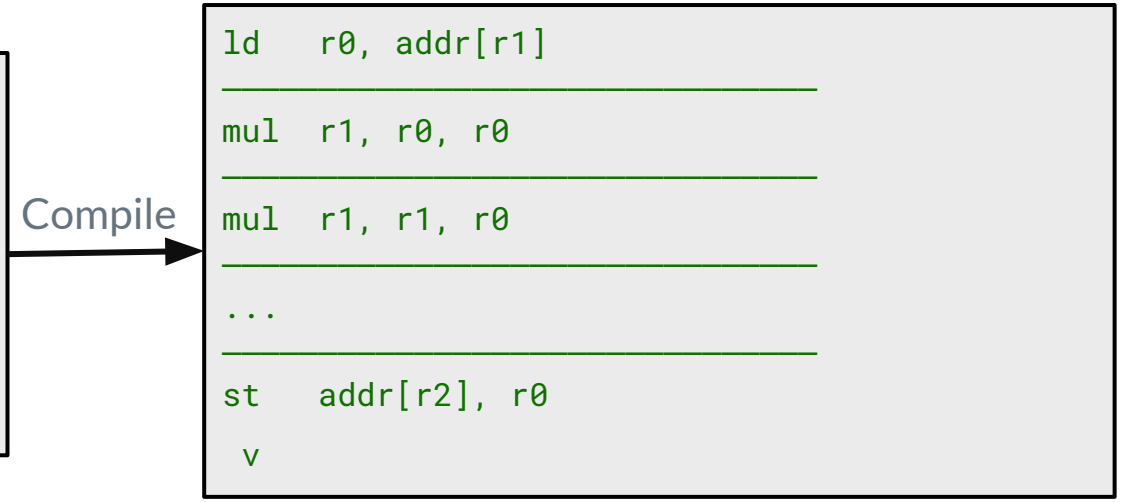
- **High complexity**: Requires searching all entries for a match (slow).
- **Impractical for large sizes**: Hardware cost grows exponentially with cache size.
- **Power-intensive**: Parallel tag comparison consumes significant energy.

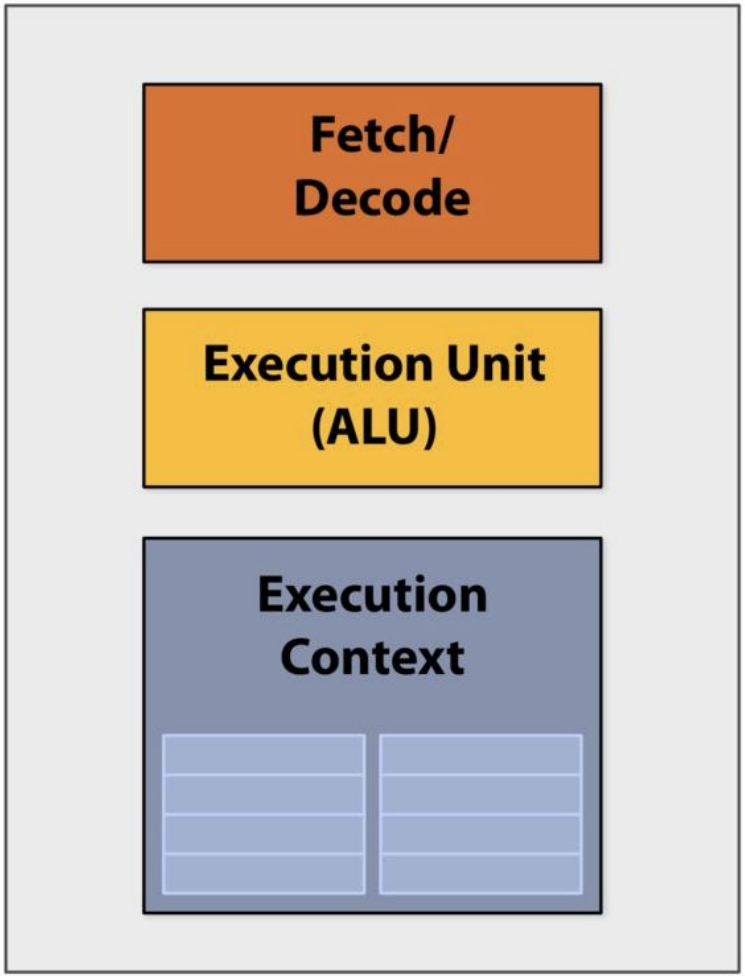


Example Program

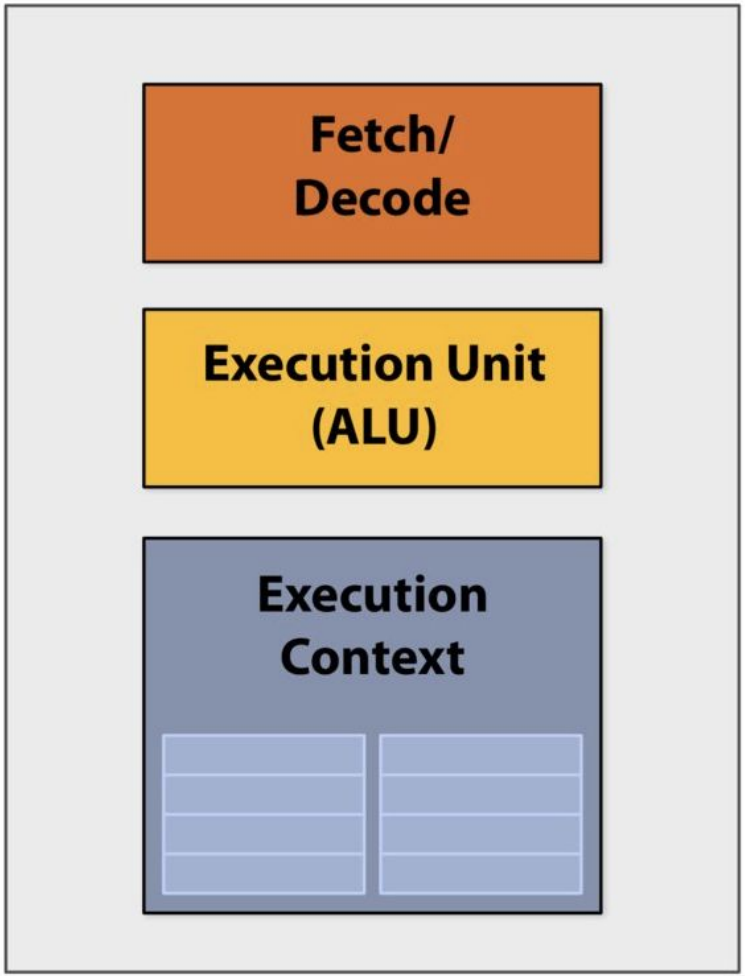
```c
int main() {
    int result = 1;
    for (int i = 0; i < 10; i++) {
        result = result * 2;
    }
    printf("%d\n", result);
    return 0;
}
```

Compile

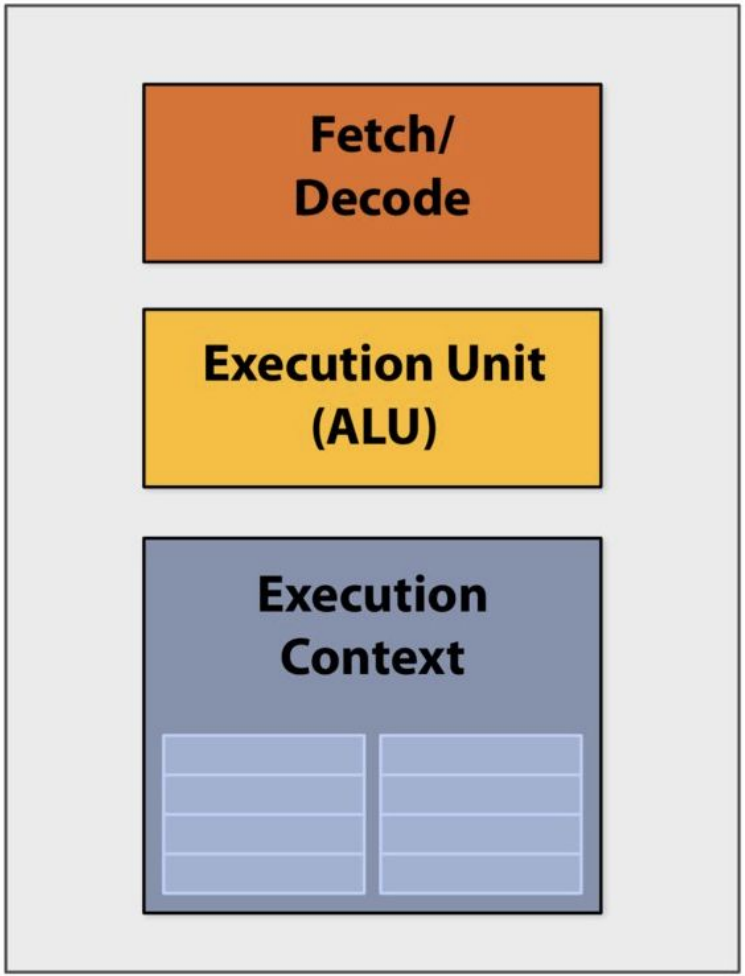




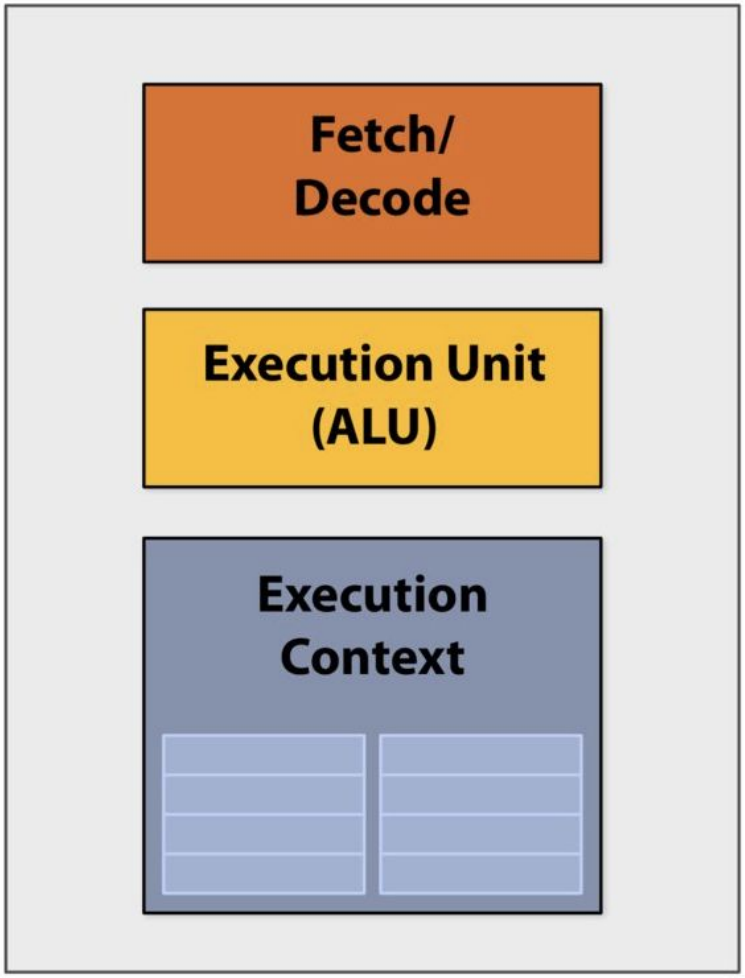

```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```




```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```




```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```




```
ld   r0, addr[r1]
mul  r1, r0, r0
mul  r1, r1, r0
...
...
...
...
...
...
st   addr[r2], r0
```

First Attempt at Parallelism (ILP)

Instruction-Level Parallelism (ILP)

Executing multiple instructions simultaneously within a single program thread.

Key Techniques

1. **Superscalar Execution**
   - Multiple instructions decoded/executed per clock cycle.
   - Example: Intel/AMD processors.
2. **Pipelining**
   - Overlap stages of instruction processing (Fetch, Decode, Execute, Writeback).
3. **Out-of-Order Execution**
   - Dynamically reorder instructions to avoid stalls.

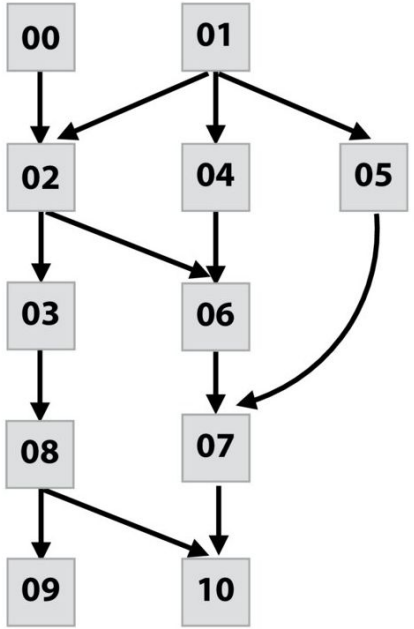
```
1. a = x + y  ──┐
2. b = z * 2    │ Independent → Can run in parallel
3. c = a + b  ──┘ (Depends on 1 and 2)
4. d = m - n  ──► Independent of 1-3
```

Example Program with ILP

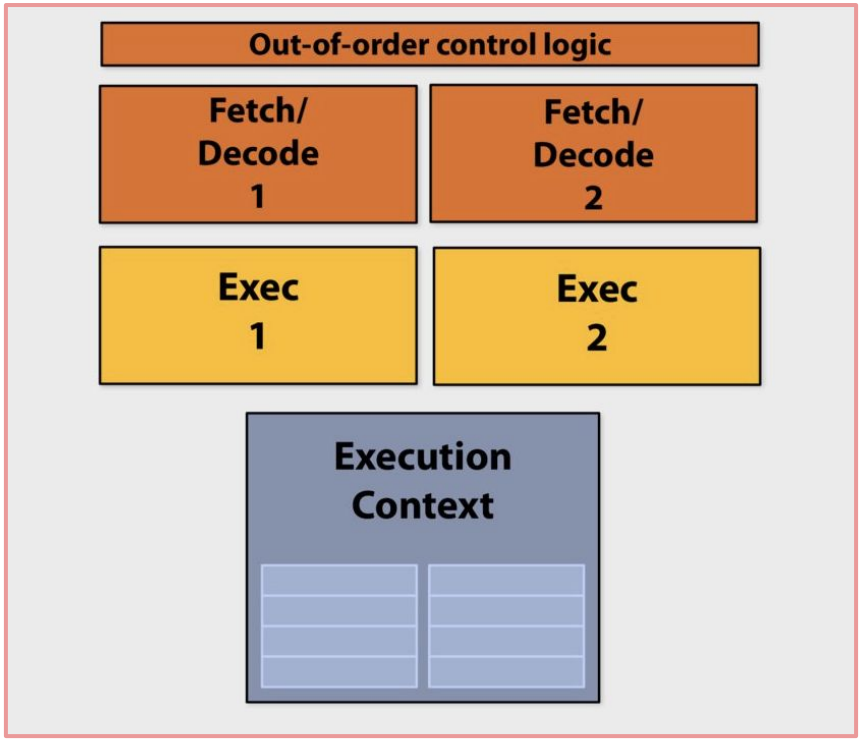
Program (sequence of instructions)

PC	Instruction	
00	a = 2	
01	b = 4	
		value during execution
02	tmp2 = a + b	// 6
03	tmp3 = tmp2 + a	// 8
04	tmp4 = b + b	// 8
05	tmp5 = b * b	// 16
06	tmp6 = tmp2 + tmp4	// 14
07	tmp7 = tmp5 + tmp6	// 30
08	if (tmp3 > 7)	
09	print tmp3	
	else	
10	print tmp7	

Instruction dependency graph



ILP: One processor, Multiple Execution Unit



Example Program

Inputs (`x`):

- A single 1D array containing **both masses and velocities** for the 2D grid:
  - The first half of `x` contains **mass values** (`x[index]`).
  - The second half contains **velocity values** (`x[index + M]`).
- Organized in a row-major format for a grid with dimensions `N x M`.

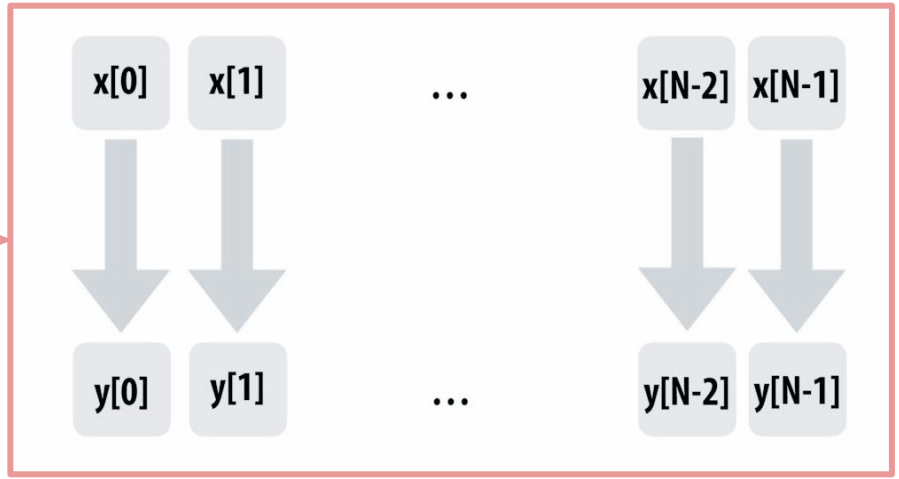
Outputs (`y`):

- A 1D array where each element represents the **total kinetic energy** for a row in the grid.
- `y[i]` corresponds to the total energy for the `i`th row.

```
1   void calculateKineticEnergy(int N, int M, float* x, float* y) {
2       for (int i = 0; i < N; i++) {
3           y[i] = 0; // Initialize energy for the current row
4
5           // Inner Loop: Sum the kinetic energy for each particle in the row
6           for (int j = 0; j < M; j++) {
7               int index = i * M + j; // Convert 2D indices to 1D index
8               float v = x[index + M]; // Velocity is stored after mass in x
9               float m = x[index];     // Mass is stored first in x
10              y[i] += 0.5 * m * v * v; // Kinetic energy = 0.5 * m * v^2
11          }
12      }
13  }
```

Example Program

```
1   void calculateKineticEnergy(int N, int M, float* x, float* y) {
2       for (int i = 0; i < N; i++) {
3           y[i] = 0; // Initialize energy for the current row
4
5           // Inner Loop: Sum the kinetic energy for each particle in the row
6           for (int j = 0; j < M; j++) {
7               int index = i * M + j; // Convert 2D indices to 1D index
8               float v = x[index + M]; // Velocity is stored after mass in x
9               float m = x[index];     // Mass is stored first in x
10              y[i] += 0.5 * m * v * v; // Kinetic energy = 0.5 * m * v^2
11          }
12      }
13  }
```

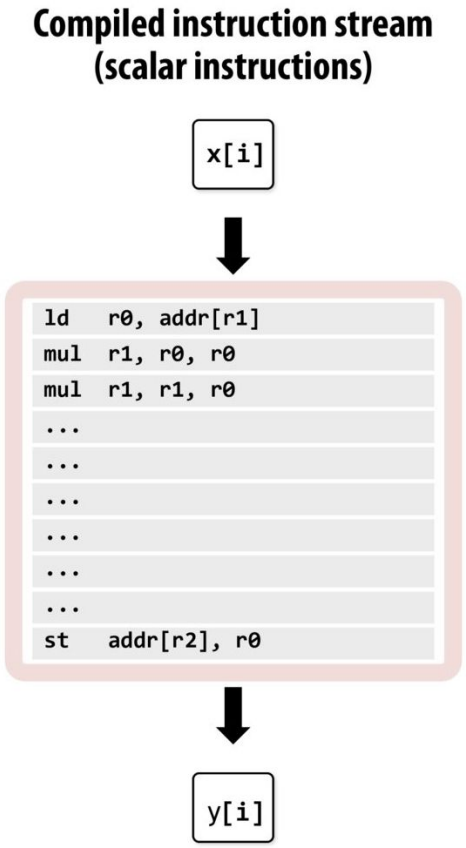


Example Program

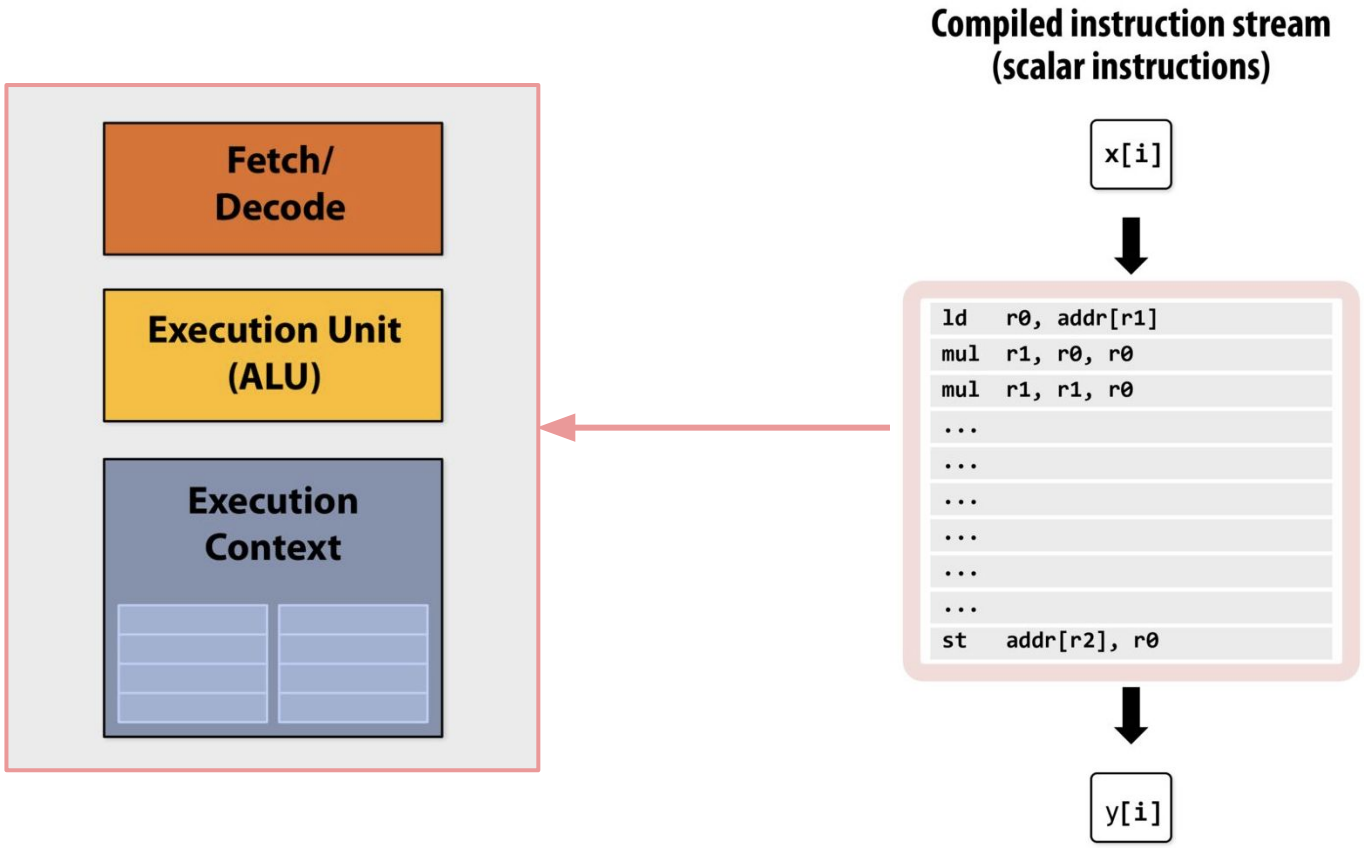
Compile the program → Sequence of Instructions

```
1   void calculateKineticEnergy(int N, int M, float* x, float* y) {
2       for (int i = 0; i < N; i++) {
3           y[i] = 0; // Initialize energy for the current row
4
5           // Inner Loop: Sum the kinetic energy for each particle in the row
6           for (int j = 0; j < M; j++) {
7               int index = i * M + j; // Convert 2D indices to 1D index
8               float v = x[index + M]; // Velocity is stored after mass in x
9               float m = x[index];     // Mass is stored first in x
10              y[i] += 0.5 * m * v * v; // Kinetic energy = 0.5 * m * v^2
11          }
12      }
13  }
```

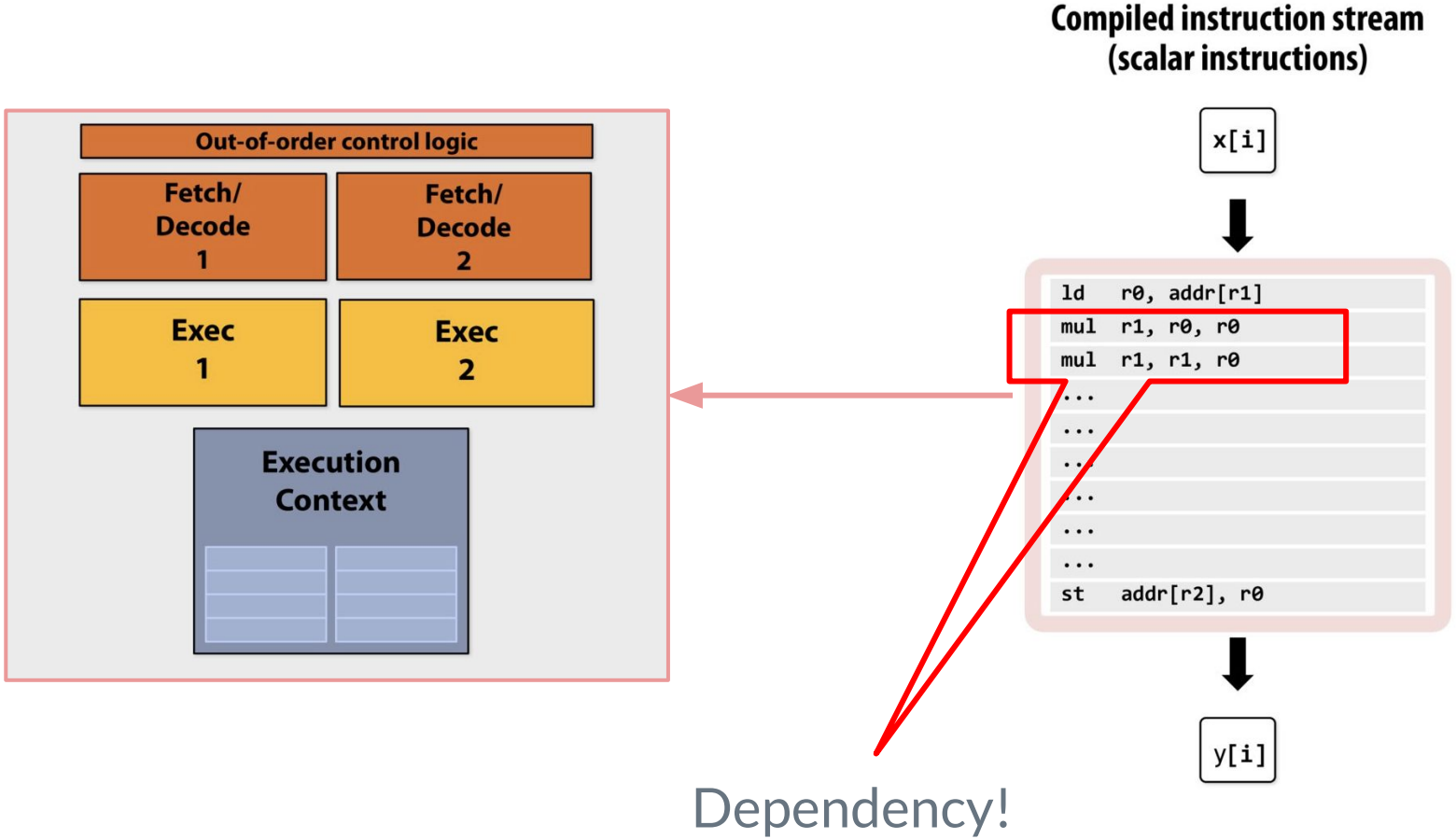
Compile



Example Program



Example Program: ILP



Dependency!

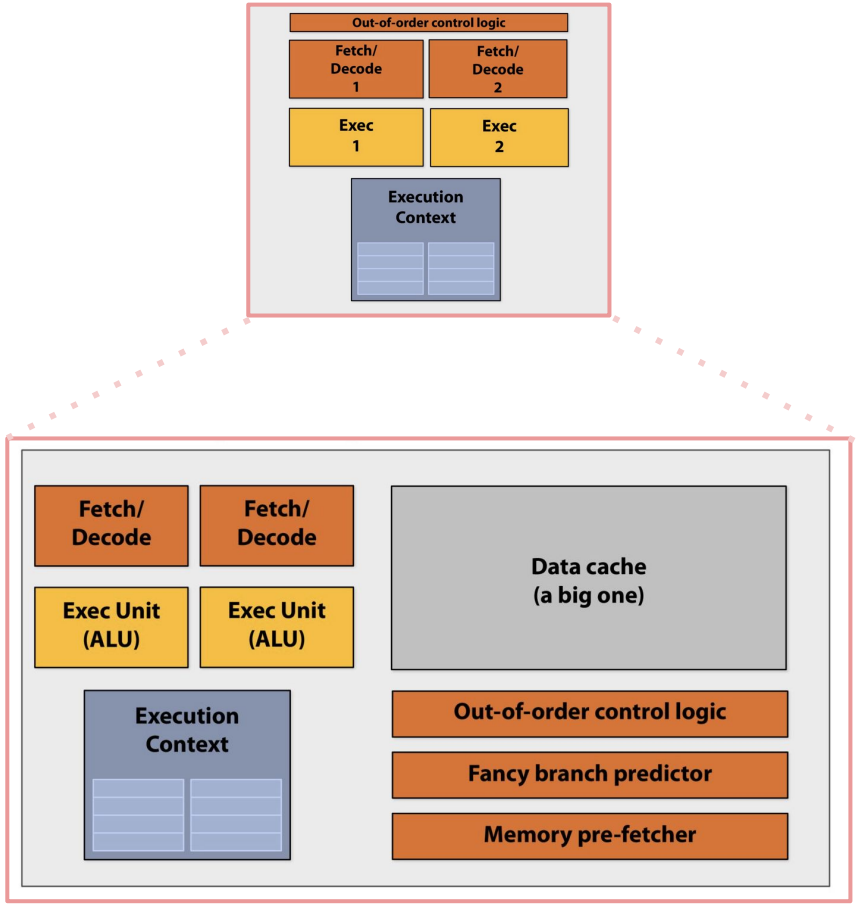
ILP vs higher level parallelism

Is there any parallelism left here?

```
1   void calculateKineticEnergy(int N, int M, float* x, float* y) {
2       for (int i = 0; i < N; i++) {
3           y[i] = 0; // Initialize energy for the current row
4
5           // Inner Loop: Sum the kinetic energy for each particle in the row
6           for (int j = 0; j < M; j++) {
7               int index = i * M + j; // Convert 2D indices to 1D index
8               float v = x[index + M]; // Velocity is stored after mass in x
9               float m = x[index];     // Mass is stored first in x
10              y[i] += 0.5 * m * v * v; // Kinetic energy = 0.5 * m * v^2
11          }
12      }
13  }
```

ILP Diminishing Return

- The majority of chip transistors are utilized to enhance the speed of executing a single instruction stream.
- **Diminishing Returns:** More transistors for ILP features yield smaller performance gains as ILP approaches its limits.
- **Higher Costs:** Complex ILP logic increases design complexity, power usage, and manufacturing costs.
- **Underutilized Potential:** Many workloads lack sufficient parallelism to fully benefit from advanced ILP optimizations.
- **Better Alternatives:** Transistors could be better spent on more cores or specialized accelerators.
- **Memory Bottlenecks:** Larger caches and smarter predictors can't fully mitigate memory access delays.
- **ILP Saturation:** Techniques like out-of-order execution face hard limits on their effectiveness.



Any other idea for parallelism?

References

- Introduction to Parallel Computing" by Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar
- Computer Architecture: A Quantitative Approach" by John L. Hennessy and David A. Patterson
- Stanford CS149
  - Highly recommended: https://www.youtube.com/watch?v=V1tINV2-9p4&list=PLoROMvodv4rMp7MTFr4hQsDEcX7Bx6Odp